

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH NÂNG CAO

ROOMHUB – HỆ THỐNG ĐẶT VÀ QUẢN LÝ
PHÒNG HỌC, PHÒNG HỌP

Nhóm sinh viên thực hiện

Nguyễn Việt Dũng	MSSV:
Hoàng Văn Tùng	MSSV:
Nguyễn Đình Phúc	MSSV:
Đỗ Thùy Linh	MSSV:

Giảng viên hướng dẫn: TS.

Mã nguồn: <https://github.com/xwomen1/classroom-booking>

HÀ NỘI, 09/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH NÂNG CAO

ROOMHUB – HỆ THỐNG ĐẶT VÀ QUẢN LÝ
PHÒNG HỌC, PHÒNG HỌP

Nhóm sinh viên thực hiện

Nguyễn Việt Dũng	MSSV: 20252690M
Hoàng Văn Tùng	MSSV: 20252389M
Nguyễn Đình Phúc	MSSV: 20251012M
Đỗ Thùy Linh	MSSV: 20252478M

Giảng viên hướng dẫn: TS. Nguyễn Thanh Bình _____

Chữ kí GVHD

Mã nguồn: <https://github.com/xwomen1/classroom-booking>

HÀ NỘI, 09/2026

LỜI CẢM ƠN

Nhóm RoomHub xin gửi lời cảm ơn chân thành tới giảng viên TS.Nguyễn Thanh Bình phụ trách học phần Lập trình nâng cao đã định hướng nội dung, góp ý về cách tổ chức mã nguồn hướng đối tượng và yêu cầu nhóm kiểm chứng sản phẩm bằng minh chứng cụ thể. Nhóm cũng cảm ơn các thầy cô Trường Điện – Điện tử, Đại học Bách khoa Hà Nội đã tạo điều kiện để nhóm khảo sát thực tế nhu cầu mượn phòng học, phòng họp trong tòa nhà C7. Những phản hồi về quy trình cấp pass, bàn giao thẻ từ và cách hiển thị vị trí phòng đã giúp nhóm điều chỉnh sản phẩm sát với nhu cầu sử dụng hơn. Cuối cùng, các thành viên cảm ơn nhau vì đã cùng phân tích nghiệp vụ, lập trình, kiểm thử trên thiết bị thật và hoàn thiện báo cáo trong thời gian thực hiện bài tập lớn.

TÓM TẮT NỘI DUNG BÀI TẬP LỚN

Nhu cầu mượn phòng học và phòng họp trong một tòa nhà tám tầng diễn ra thường xuyên, nhưng việc đăng ký hiện vẫn dựa vào trao đổi thủ công. Người mượn khó biết phòng nào còn trống theo khung giờ, sức chứa và thiết bị mong muốn, không rõ vị trí phòng trên từng tầng, trong khi cán bộ cơ sở vật chất phải tự đối chiếu lịch, bàn giao thẻ từ hoặc thay đổi mật khẩu khóa tại cửa. Các công cụ lịch dùng chung giải quyết được việc ghi lịch nhưng không kiểm soát quy tắc nghiệp vụ như hạn đặt trước, giới hạn số yêu cầu, lịch bảo trì hay quyền tiếp cận phòng sau khi được duyệt.

Nhóm lựa chọn hướng xây dựng một hệ thống hướng đối tượng gồm ứng dụng di động React Native viết bằng TypeScript và dịch vụ nghiệp vụ Java dùng Spring Boot, JPA và cơ sở dữ liệu H2. Hướng tiếp cận này cho phép tách rõ tầng giao diện, tầng dịch vụ chứa quy tắc nghiệp vụ và tầng dữ liệu, đồng thời kiểm tra lại mọi thao tác ở phía máy chủ. Sản phẩm mang tên RoomHub hỗ trợ hai vai trò Admin và User; người dùng tìm phòng trên sơ đồ chữ U theo tầng, lọc theo thời gian, sức chứa, thiết bị, loại khóa rồi gửi yêu cầu ngay trên cùng màn hình. Quản trị viên duyệt, từ chối, đổi phòng, quản lý tài khoản, phòng, lịch bảo trì và các tham số nghiệp vụ.

Đóng góp chính của bài tập lớn gồm (i) bộ quy tắc kiểm tra đặt phòng dựa trên phép giao khoảng thời gian kết hợp khóa bí quan trong giao dịch để chống trùng lịch, (ii) cơ chế cấp quyền tạo mã PIN theo cặp người dùng – phòng và gửi mật khẩu tạm thời có thời hạn tới khóa thông minh qua gateway MQTT, và (iii) kiến trúc module cho phép chuyển giữa chế độ lưu cục bộ và đồng bộ máy chủ bằng một cờ cấu hình. Ứng dụng đã được đóng gói thành APK Release và chạy độc lập trên điện thoại Samsung M21; biên bản kiểm tra ngày 26/09/2026 ghi nhận 13/13 ca kiểm thử tự động đạt, sau đó nhóm bổ sung thêm ca kiểm thử cho liên kết khóa thông minh.

Nhóm sinh viên thực hiện

(Ký và ghi rõ họ tên)

ABSTRACT

Classrooms and meeting rooms in an eight-floor university building are borrowed every day, yet reservations are still handled manually. Requesters cannot easily see which rooms are free for a given time slot, capacity and equipment, nor where a room is located on each floor, while facility staff must reconcile schedules by hand, hand over access cards or change door passwords on site. Shared calendars record events but do not enforce business rules such as the booking window, per-user limits, maintenance periods or the access rights granted after approval.

The team chose an object-oriented solution consisting of a React Native mobile application written in TypeScript and a Java business service built with Spring Boot, JPA and an H2 database. This approach separates the presentation, service and persistence layers and re-validates every operation on the server. The resulting product, RoomHub, supports two roles. Users search rooms on a U-shaped floor map, filter by time, capacity, equipment and lock type, and submit a request on the same screen. Administrators approve, reject or relocate bookings and manage accounts, rooms, maintenance schedules and business parameters.

The main contributions are (i) a booking rule engine based on time-interval intersection combined with pessimistic locking inside a transaction to prevent double booking, (ii) a room-scoped permission model that lets an approved user generate a time-limited PIN which is delivered to a smart lock through an MQTT gateway, and (iii) a modular architecture that switches between local storage and server synchronisation with a single configuration flag. The application has been packaged as a release APK and runs standalone on a Samsung M21 phone; the device verification record of 26 September 2026 reports 13 out of 13 automated test cases passing, and a further test case for the smart lock binding was added afterwards.

PHÂN CÔNG NHIỆM VỤ

Bảng dưới đây tổng hợp phân công công việc và đóng góp chính của các thành viên trong quá trình thực hiện đề tài RoomHub.

Bảng 0.1: Phân công công việc trong nhóm

Thành viên	Phần việc chính
Đỗ Thùy Linh	Xây dựng dịch vụ Booking Server bằng Spring Boot; cấu hình cơ sở dữ liệu H2 từ xa; phát triển banner thông báo ưu tiên; thiết kế logo; xây dựng chức năng ẩn/hiện mật khẩu.
Hoàng Văn Tùng	Xây dựng khung ứng dụng và kiến trúc module; phát triển phần lớn giao diện và nghiệp vụ; tích hợp OneIoT và chức năng PIN tạm thời; xây dựng mô phỏng SmartLock và gateway; thực hiện build Release.
Nguyễn Việt Dũng	Xây dựng quy trình và các kịch bản kiểm thử; thực hiện kiểm thử công toàn bộ các luồng chức năng; tổng hợp và phân loại 14 nhóm lỗi của hệ thống.Xây dựng hệ thống nghiệp vụ và tài liệu dự án.
Nguyễn Đình Phúc	Phát triển chức năng đặt phòng lặp hàng tuần và quản lý chuỗi đặt phòng; xây dựng cơ chế thông báo phê duyệt; quản lý trường hợp no-show; xây dựng kiểm thử tự động cho chức năng booking.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục báo cáo	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	4
2.1 Khảo sát hiện trạng	4
2.1.1 Khảo sát người dùng và quy trình hiện hành	4
2.1.2 Khảo sát các giải pháp tương tự	5
2.2 Tổng quan chức năng	6
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã “Tìm và đặt phòng”	7
2.2.3 Biểu đồ use case phân rã “Duyệt / từ chối / đổi phòng”	7
2.2.4 Quy trình nghiệp vụ	8
2.3 Đặc tả chức năng	10
2.3.1 Đặc tả use case “Đăng nhập”	12
2.3.2 Đặc tả use case “Tìm và đặt phòng”	13
2.3.3 Đặc tả use case “Duyệt yêu cầu đặt phòng”	14
2.3.4 Đặc tả use case “Đổi phòng thay thế”	15
2.3.5 Đặc tả use case “Tạo mã PIN tạm thời”	16
2.4 Yêu cầu phi chức năng	16
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	18
3.1 Lập trình hướng đối tượng với Java 17	18
3.2 Spring Boot và cơ chế đảo ngược điều khiển	19

3.3 Jakarta Persistence, Hibernate và cơ sở dữ liệu H2	19
3.4 Kiến trúc REST và định dạng JSON	20
3.5 React Native và TypeScript.....	20
3.6 Giao thức MQTT, chuẩn oneM2M và mã hóa AES	21
3.7 Mô hình hóa UML và kiểm thử tự động	21
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	23
4.1 Thiết kế kiến trúc.....	23
4.1.1 Lựa chọn kiến trúc phần mềm	23
4.1.2 Thiết kế tổng quan.....	24
4.1.3 Thiết kế chi tiết gói	25
4.2 Thiết kế chi tiết.....	26
4.2.1 Thiết kế giao diện	26
4.2.2 Thiết kế lớp	27
4.2.3 Thiết kế cơ sở dữ liệu	30
4.3 Xây dựng ứng dụng.....	32
4.3.1 Thư viện và công cụ sử dụng.....	32
4.3.2 Kết quả đạt được	33
4.3.3 Minh họa các chức năng chính	34
4.4 Kiểm thử.....	36
4.5 Triển khai	38
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	39
5.1 Chống trùng lịch bằng phép giao khoảng thời gian và khóa bí quan.....	39
5.1.1 Vấn đề.....	39
5.1.2 Giải pháp	39
5.1.3 Kết quả đạt được	42

5.2 Cấp quyền tạo mã PIN theo phòng và tích hợp khóa thông minh	43
5.2.1 Vấn đề.....	43
5.2.2 Giải pháp	43
5.2.3 Kết quả đạt được	45
5.3 Kiến trúc module hai chế độ dữ liệu	45
5.3.1 Vấn đề.....	45
5.3.2 Giải pháp	46
5.3.3 Kết quả đạt được	47
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	48
6.1 Kết luận	48
6.2 Hướng phát triển.....	48
TÀI LIỆU THAM KHẢO.....	50
PHỤ LỤC.....	53
A. ĐẶC TẢ USE CASE BỔ SUNG.....	53
A.1 Đặc tả use case “Hủy đặt phòng”	53
A.2 Đặc tả use case “Hẹn lịch nhận khóa và ủy quyền nhận hộ”	54
A.3 Đặc tả use case “Quản lý lịch sử dụng và bảo trì”.....	54
A.4 Đặc tả use case “Cấu hình nghiệp vụ”	55
A.5 Đặc tả use case “Quản lý SmartLock”	55
A.6 Đặc tả use case “Quản lý tài khoản và quyền theo phòng”	56
B. DANH SÁCH API, CÀI ĐẶT VÀ MINH CHỨNG KIỂM THỬ.....	57
B.1 Danh sách REST API của dịch vụ Java	57
B.2 Hướng dẫn cài đặt và chạy thử.....	57
B.3 Biên bản kiểm tra trên thiết bị	58

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống RoomHub	6
Hình 2.2	Biểu đồ use case phân rã “Tìm và đặt phòng”	7
Hình 2.3	Biểu đồ use case phân rã “Duyệt / từ chối / đổi phòng”	8
Hình 2.4	Biểu đồ hoạt động của quy trình đặt phòng và cấp quyền tiếp cận	9
Hình 2.5	Biểu đồ trạng thái của booking và của mã PIN tạm thời	10
Hình 4.1	Kiến trúc tổng thể của hệ thống RoomHub	23
Hình 4.2	Biểu đồ gói của ứng dụng di động RoomHub	24
Hình 4.3	Màn hình đăng nhập và bảng điều khiển theo vai trò	26
Hình 4.4	Biểu đồ lớp của dịch vụ nghiệp vụ Java booking-server	27
Hình 4.5	Biểu đồ trình tự ca sử dụng “Tìm và đặt phòng” qua dịch vụ Java	29
Hình 4.6	Biểu đồ trình tự ca sử dụng “Duyệt yêu cầu” và “Tạo mã PIN tạm thời”	30
Hình 4.7	Biểu đồ thực thể liên kết của cơ sở dữ liệu RoomHub	31
Hình 4.8	Màn hình tìm phòng theo sơ đồ tầng và gửi yêu cầu đặt phòng	34
Hình 4.9	Màn hình quản lý phòng và lịch bảo trì của Admin	35
Hình 4.10	Màn hình quản lý quyền theo phòng và cấu hình tham số nghiệp vụ	36

DANH MỤC BẢNG BIỂU

Bảng 0.1	Phân công công việc trong nhóm	
Bảng 2.1	Cấu hình phòng theo phương thức tiếp cận trong quy trình hiện hành	4
Bảng 2.2	So sánh các nhóm giải pháp quản lý đặt phòng	5
Bảng 2.3	Các quy tắc nghiệp vụ đã hiện thực trong RoomHub	11
Bảng 2.4	Đặc tả use case UC01 – Đăng nhập	12
Bảng 2.5	Đặc tả use case UC02 – Tìm và đặt phòng	13
Bảng 2.6	Đặc tả use case UC03 – Duyệt yêu cầu đặt phòng	14
Bảng 2.7	Đặc tả use case UC04 – Đổi phòng thay thế	15
Bảng 2.8	Đặc tả use case UC05 – Tạo mã PIN tạm thời	16
Bảng 3.1	Tổng hợp lựa chọn công nghệ theo yêu cầu	22
Bảng 4.1	Nhiệm vụ các module nghiệp vụ của ứng dụng di động	25
Bảng 4.2	Thiết kế chi tiết lớp BookingService	28
Bảng 4.3	Cấu trúc bảng bookings	32
Bảng 4.4	Danh sách thư viện và công cụ sử dụng	33
Bảng 4.5	Thông kê mã nguồn và sản phẩm đóng gói	34
Bảng 4.6	Ca kiểm thử chức năng tạo yêu cầu đặt phòng	36
Bảng 4.7	Ca kiểm thử chức năng duyệt yêu cầu và tạo mã PIN	37
Bảng 4.8	Ca kiểm thử chức năng quản trị và phân quyền	37
Bảng 5.1	Đối chiếu tăng dịch vụ TypeScript và dịch vụ Java	47
Bảng A.1	Đặc tả use case UC06 – Hủy đặt phòng	53
Bảng A.2	Đặc tả use case UC07 – Hẹn lịch nhận khóa và ủy quyền nhận hộ	54
Bảng A.3	Đặc tả use case UC08 – Quản lý lịch sử dụng và bảo trì	54
Bảng A.4	Đặc tả use case UC09 – Cấu hình nghiệp vụ	55
Bảng A.5	Đặc tả use case UC10 – Quản lý SmartLock	55
Bảng A.6	Đặc tả use case UC11 – Quản lý tài khoản và quyền theo phòng	56
Bảng B.1	Danh sách REST API của booking-server	57
Bảng B.2	Trích biên bản kiểm tra bản Release trên Samsung M21	59

DANH MỤC ĐOẠN MÃ

4.1	Khai báo thực thể Booking với TemporaryPin nhúng (rút gọn) . . .	28
5.1	Hàm giao khoảng và kiểm tra xung đột bảo trì trong BookingService	40
5.2	Tuần tự hóa thao tác tạo booking bằng khóa bí quan	42
5.3	Tính khoảng hiệu lực và gửi lệnh tạo mã (temporaryPinService.ts, rút gọn)	44
5.4	Một hàm repository hai chế độ (bookingRepository.ts, rút gọn) . . .	46

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AES	Chuẩn mã hóa nâng cao (Advanced Encryption Standard)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
APK	Gói cài đặt ứng dụng Android (Android Package Kit)
BR	Quy tắc nghiệp vụ (Business Rule)
CRUD	Tạo, đọc, cập nhật, xóa (Create, Read, Update, Delete)
CSVC	Cơ sở vật chất
CTR	Chế độ mã hóa bộ đếm (Counter mode)
DI	Tiêm phụ thuộc (Dependency Injection)
DTO	Đối tượng truyền dữ liệu (Data Transfer Object)
HTTP	Giao thức truyền siêu văn bản (HyperText Transfer Protocol)
IoC	Đảo ngược điều khiển (Inversion of Control)
IoT	Internet vạn vật (Internet of Things)
JPA	Đặc tả lưu trữ Jakarta (Jakarta Persistence API)
JSON	Định dạng dữ liệu JavaScript Object Notation
JVM	Máy ảo Java (Java Virtual Machine)
MQTT	Giao thức nhắn tin xuất bản – đăng ký cho thiết bị IoT (Message Queuing Telemetry Transport)
MVP	Sản phẩm khả dụng tối thiểu (Minimum Viable Product)
ORM	Ánh xạ đối tượng – quan hệ (Object-Relational Mapping)
PIN	Mã số cá nhân dùng để mở khóa (Personal Identification Number)
QoS	Chất lượng dịch vụ (Quality of Service)
REST	Kiểu kiến trúc chuyển trạng thái đại diện (Representational State Transfer)
TLS	Giao thức bảo mật tầng giao vận (Transport Layer Security)
UML	Ngôn ngữ mô hình hóa thống nhất (Unified Modeling Language)
UUID	Định danh duy nhất toàn cục (Universally Unique Identifier)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong một trường đại học, phòng học và phòng họp là tài nguyên dùng chung có tần suất sử dụng cao. Ngoài thời khóa biểu cố định, giảng viên thường xuyên cần mượn thêm phòng để dạy bù, tổ chức seminar, họp bộ môn hoặc hướng dẫn nhóm đồ án. Với tòa nhà C7 mà nhóm khảo sát, mỗi tầng có bảy phòng bố trí theo sơ đồ chữ U, phòng số lẻ nằm một phía hành lang, phòng số chẵn nằm phía đối diện, xen giữa là thang máy, cầu thang và khu vệ sinh; phần lớn người dùng di chuyển bằng thang máy nên việc biết trước vị trí phòng trên từng tầng giúp tiết kiệm đáng kể thời gian đi lại.

Hiện nay quy trình mượn phòng chủ yếu dựa vào trao đổi trực tiếp, điện thoại hoặc tin nhắn với cán bộ cơ sở vật chất (CSVC). Cách làm này bộc lộ bốn hạn chế. Thứ nhất, người mượn khó biết phòng nào còn trống trong khung giờ mong muốn. Thứ hai, thông tin sức chứa và trang thiết bị như máy chiếu, micro, điều hòa không có sẵn để tra cứu nhanh. Thứ ba, người mượn không rõ phòng nằm ở phía nào của tầng. Thứ tư, các thông báo về kết quả duyệt, lịch nhận khóa hay thay đổi phòng không đến kịp thời. Ở phía quản lý, cán bộ CSVC phải tự đối chiếu lịch để tránh trùng, đồng thời xử lý hai phương thức tiếp cận phòng khác nhau là bàn giao thẻ từ và cấp mật khẩu (pass) cho khóa mã số.

Nếu bài toán được giải quyết, giảng viên có thể tự tra cứu và gửi yêu cầu trong vài thao tác, cán bộ CSVC có công cụ duyệt tập trung với quy tắc được kiểm tra tự động, và nhà trường có dữ liệu để đối soát lịch sử sử dụng phòng. Mô hình này cũng áp dụng được cho các tài nguyên đặt trước khác như phòng thí nghiệm, hội trường hoặc thiết bị dùng chung.

1.2 Mục tiêu và phạm vi đề tài

Các công cụ đang được dùng để quản lý lịch phòng có thể chia thành ba nhóm. Nhóm thứ nhất là sổ ghi chép hoặc bảng tính dùng chung, dễ triển khai nhưng không ngăn được trùng lịch và không phân quyền. Nhóm thứ hai là lịch trực tuyến như Google Calendar hay Microsoft Outlook với tài nguyên phòng; các công cụ này hiển thị được lịch bận nhưng không mô hình hóa quy tắc riêng của nhà trường như hạn đặt trước từ một đến ba ngày, giới hạn số yêu cầu đang hoạt động của mỗi người hoặc lịch bảo trì. Nhóm thứ ba là các phần mềm quản lý cơ sở vật chất thương mại, có đầy đủ chức năng nhưng chi phí cao và không gắn với loại khóa mà tòa nhà đang sử dụng. Điểm chung của các nhóm trên là tách rời việc *đặt phòng* với việc *tiếp cận phòng*: sau khi được duyệt, người mượn vẫn phải liên hệ riêng để

nhận thẻ hoặc mật khẩu.

Xuất phát từ các hạn chế đó, bài tập lớn đặt mục tiêu xây dựng hệ thống RoomHub gồm ứng dụng di động cho người dùng cuối và dịch vụ nghiệp vụ Java. Phạm vi của đề tài bao gồm (i) đăng nhập, đăng ký và điều hướng theo hai vai trò Admin và User; (ii) tìm phòng trên sơ đồ tầng theo ngày giờ, sức chứa, thiết bị và loại khóa, rồi gửi yêu cầu ngay trên cùng màn hình; (iii) duyệt, từ chối và đổi phòng thay thế có kiểm tra điều kiện; (iv) quản lý tài khoản, phòng, lịch bảo trì và các tham số nghiệp vụ; (v) cấp quyền tiếp cận theo loại khóa, gồm hẹn lịch nhận khóa cơ hoặc thẻ từ và tạo mã PIN tạm thời cho khóa mã số; (vi) thông báo theo sự kiện. Các nghiệp vụ check-in bằng QR, xử lý no-show, audit log đầy đủ và báo cáo thống kê được mô tả trong quy trình nghiệp vụ nhưng nằm ngoài phạm vi phiên bản hiện tại.

1.3 Định hướng giải pháp

Nhóm tiếp cận bài toán theo phương pháp phân tích thiết kế hướng đối tượng. Phía máy chủ sử dụng Java 17 với Spring Boot 3.4 để xây dựng dịch vụ REST, Jakarta Persistence (JPA) để ánh xạ các lớp thực thể xuống cơ sở dữ liệu H2. Lựa chọn này phù hợp với yêu cầu của học phần Lập trình nâng cao về lập trình Java, đồng thời tận dụng cơ chế tiêm phụ thuộc, quản lý giao dịch và xử lý ngoại lệ tập trung của Spring. Phía người dùng là ứng dụng React Native viết bằng TypeScript để chạy trực tiếp trên điện thoại Android, nơi giảng viên thường thao tác khi di chuyển giữa các tầng.

Giải pháp tổ chức mã nguồn thành các module nghiệp vụ độc lập (xác thực, phòng, đặt phòng, kiểm soát truy cập, khóa thông minh, bảo trì, cấu hình, thông báo). Mọi quy tắc đặt phòng được đặt trong tầng dịch vụ và được kiểm tra lại ở máy chủ trong một giao dịch có khóa bi quan. Với phòng dùng khóa mã số, việc duyệt yêu cầu tự động cấp quyền cho đúng cặp người dùng – phòng; người dùng tự tạo mã PIN sáu chữ số, ứng dụng gửi lệnh tới gateway để gateway mã hóa và phát bản tin MQTT tới khóa DLWA12 qua nền tảng OneIoT.

Kết quả đạt được là một sản phẩm chạy được trên thiết bị thật: APK Release độc lập không cần máy tính, mã nguồn di động khoảng 5.200 dòng TypeScript, dịch vụ Java 860 dòng với tám lớp thực thể, cùng bộ kiểm thử tự động cho các quy tắc chính. Các đóng góp nổi bật được trình bày chi tiết tại Chương 5.

1.4 Bố cục báo cáo

Phần còn lại của báo cáo được tổ chức như sau.

Chương 2 trình bày kết quả khảo sát hiện trạng mượn phòng và tài liệu quy trình

ng nghiệp vụ, từ đó xây dựng biểu đồ use case tổng quát, các biểu đồ phân rã, quy trình nghiệp vụ đặt phòng, đặc tả năm use case quan trọng và các yêu cầu phi chức năng.

Chương 3 giới thiệu nền tảng lý thuyết và công nghệ sử dụng, gồm lập trình hướng đối tượng với Java, Spring Boot và JPA, kiến trúc REST, React Native với TypeScript, giao thức MQTT và mã hóa AES; với mỗi công nghệ, báo cáo nêu yêu cầu mà công nghệ đó giải quyết và lý do lựa chọn so với phương án thay thế.

Chương 4 trình bày thiết kế kiến trúc, thiết kế gói, thiết kế lớp, biểu đồ trình tự, thiết kế cơ sở dữ liệu và giao diện, sau đó mô tả kết quả xây dựng ứng dụng, các ca kiểm thử và mô hình triển khai trên thiết bị thật.

Chương 5 tập trung vào ba đóng góp chính: thuật toán kiểm tra xung đột lịch kết hợp khóa bí quan, mô hình cấp quyền tạo mã PIN theo phòng tích hợp khóa thông minh, và kiến trúc module hai chế độ dữ liệu.

Chương 6 tổng kết kết quả, so sánh với các giải pháp hiện có, nêu hạn chế và hướng phát triển.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã nêu bài toán mượn phòng học, phòng họp và phạm vi của hệ thống RoomHub. Chương này trình bày kết quả khảo sát hiện trạng từ ba nguồn là người dùng, quy trình đang áp dụng và các công cụ tương tự; từ đó xác định các tác nhân, biểu đồ use case tổng quát và phân rã, quy trình nghiệp vụ, đặc tả các use case quan trọng cùng bộ quy tắc nghiệp vụ và yêu cầu phi chức năng. Đây là cơ sở để lựa chọn công nghệ ở Chương 3 và thiết kế ở Chương 4.

2.1 Khảo sát hiện trạng

2.1.1 Khảo sát người dùng và quy trình hiện hành

Nguồn khảo sát thứ nhất là trao đổi với giảng viên và cán bộ quản lý cơ sở vật chất của tòa nhà tám tầng. Nhu cầu mượn phòng phát sinh thường xuyên nhưng thiếu công cụ trực quan; người mượn cần biết đồng thời bốn thông tin là phòng còn trống theo khung giờ, sức chứa, trang thiết bị và vị trí phòng trên tầng. Do đa số người dùng đi thang máy, sơ đồ tầng cần thể hiện rõ phòng số lẻ và số chẵn nằm ở hai phía hành lang để người dùng tìm đúng vị trí.

Nguồn thứ hai là tài liệu *Quy trình nghiệp vụ đặt phòng học/phòng họp* phiên bản 1.1 ngày 18/09/2026 do nhóm xây dựng cùng đơn vị quản lý phòng [1]. Tài liệu mô tả quy trình tổng thể gồm tám bước: tra cứu phòng, chọn phòng và thời gian, gửi yêu cầu, phê duyệt, chuẩn bị tiếp cận, check-in và sử dụng, check-out và trả thẻ, đóng booking. Điểm đặc thù của hiện trạng là phòng được tiếp cận theo hai phương thức: *thẻ từ* được bàn giao tại quầy CSVC và *pass* của khóa mã số. Loại khóa mã số cũ không có kết nối điều khiển từ xa nên nhân viên phải lên tận nơi để đặt hoặc đổi mật khẩu. Bảng 2.1 tóm tắt bốn cấu hình phòng theo phương thức tiếp cận được nêu trong tài liệu.

Bảng 2.1: Cấu hình phòng theo phương thức tiếp cận trong quy trình hiện hành

Loại	Phương thức	Quy trình tiếp cận
Loại A	Chỉ dùng pass	Nhân viên CSVC đặt pass tại khóa trước giờ sử dụng và đổi pass sau khi kết thúc.
Loại B	Chỉ dùng thẻ từ	Người dùng nhận thẻ tại điểm bàn giao, sử dụng và trả thẻ sau khi kết thúc.
Loại C	Có cả pass và thẻ, booking dùng pass	Áp dụng quy trình cấp pass, không mặc nhiên bàn giao thẻ.
Loại D	Có cả pass và thẻ, booking dùng thẻ	Áp dụng quy trình nhận/trả thẻ, không mặc nhiên cấp pass.

Trong quá trình khảo sát, nhóm được tiếp cận một khóa thông minh DLWA12 kết nối nền tảng OneIoT, cho phép tạo mật khẩu tạm thời từ xa bằng bản tin MQTT. Vì vậy, RoomHub khái quát bốn cấu hình trên thành hai loại khóa: *khóa cơ/thẻ từ* (PHYSICAL_KEY), tương ứng nghiệp vụ hẹn lịch bàn giao, và *khóa mã số quản lý online* (PIN_CODE), tương ứng nghiệp vụ cấp mật khẩu tạm thời có thời hạn. Cách khái quát này giữ nguyên nguyên tắc của tài liệu nghiệp vụ là mỗi booking chỉ dùng một phương thức tiếp cận, đồng thời loại bỏ bước nhân viên phải lên tận khóa đối với phòng đã gắn khóa thông minh.

2.1.2 Khảo sát các giải pháp tương tự

Nguồn thứ ba là các công cụ đang được dùng để quản lý lịch phòng. Bảng 2.2 so sánh bốn nhóm giải pháp theo các tiêu chí rút ra từ khảo sát người dùng.

Bảng 2.2: So sánh các nhóm giải pháp quản lý đặt phòng

Tiêu chí	Sổ ghi / bảng tính	Lịch tài nguyên trực tuyến	Phần mềm đặt chỗ thương mại	RoomHub
Trạng thái phòng theo khung giờ	Thủ công	Có	Có	Có
Sơ đồ tầng, vị trí phòng	Không	Không	Thường có	Có (chữ U, 8 tầng)
Lọc sức chứa, thiết bị	Không	Hạn chế	Có	Có
Quy tắc riêng (hạn đặt trước, giới hạn yêu cầu, bảo trì)	Không	Hạn chế	Có, cấu hình theo gói	Có, Admin chỉnh trong ứng dụng
Quy trình phê duyệt	Thủ công	Tự động chấp nhận hoặc từ chối	Có	Có, kèm đổi phòng
Cấp quyền tiếp cận theo loại khóa	Không	Không	Qua tích hợp riêng	Hẹn nhận khóa / mã PIN tạm thời
Chi phí, khả năng tùy biến	Thấp, không kiểm soát	Theo gói dịch vụ	Thuê bao cao	Tự phát triển

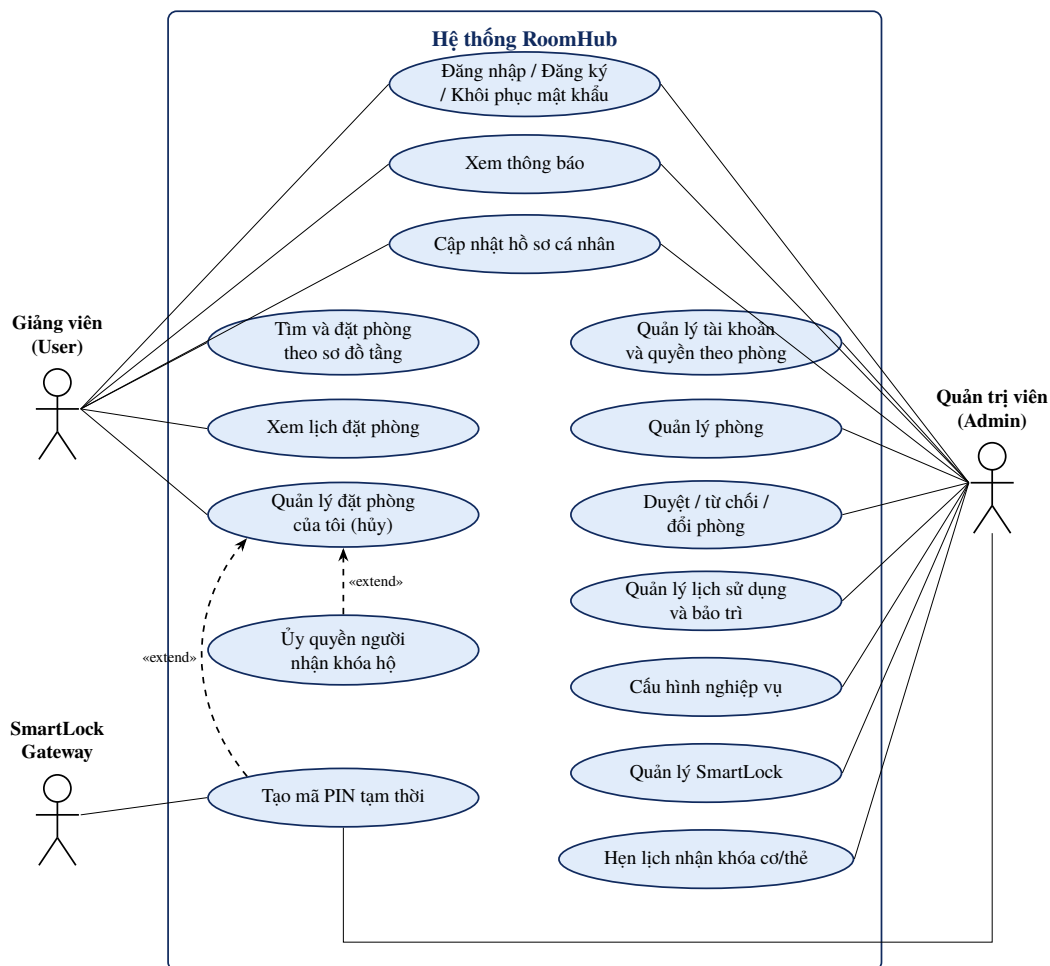
Kết quả so sánh cho thấy các công cụ phổ biến giải quyết tốt việc ghi lịch nhưng chưa gắn quy tắc riêng của nhà trường và chưa nối liền bước đặt phòng với bước tiếp cận phòng. Từ đó, nhóm xác định các tính năng quan trọng cần phát triển gồm: tìm phòng trực quan trên sơ đồ tầng với màu trạng thái trống, đã đặt, bảo trì; gửi yêu cầu ngay tại phòng đã chọn; kiểm tra tự động các quy tắc đặt phòng; phê duyệt và đổi phòng có điều kiện; cấp quyền tiếp cận theo loại khóa; thông báo kịp thời; và cho phép quản trị viên chỉnh tham số nghiệp vụ mà không phải sửa mã nguồn.

2.2 Tổng quan chức năng

Mục này tóm tắt chức năng của hệ thống ở mức tổng quan; đặc tả chi tiết được trình bày trong mục 2.3.

2.2.1 Biểu đồ use case tổng quát

Hệ thống có ba tác nhân. **Giảng viên (User)** là người mượn phòng, có nhu cầu tra cứu, đặt, theo dõi và hủy yêu cầu, ủy quyền người nhận khóa hộ và tạo mã PIN cho phòng khóa số. **Quản trị viên (Admin)** là cán bộ CSVN, chịu trách nhiệm quản lý tài khoản, phòng, lịch bảo trì, cấu hình nghiệp vụ, khóa thông minh và xử lý các yêu cầu đặt phòng. **SmartLock Gateway** là tác nhân hệ thống ngoài, nhận lệnh tạo mật khẩu tạm thời và chuyển tới khóa. Hình 2.1 thể hiện biểu đồ use case tổng quát. Người dùng không tự chọn vai trò khi đăng nhập; hệ thống đọc vai trò từ tài khoản và mở đúng giao diện tương ứng.



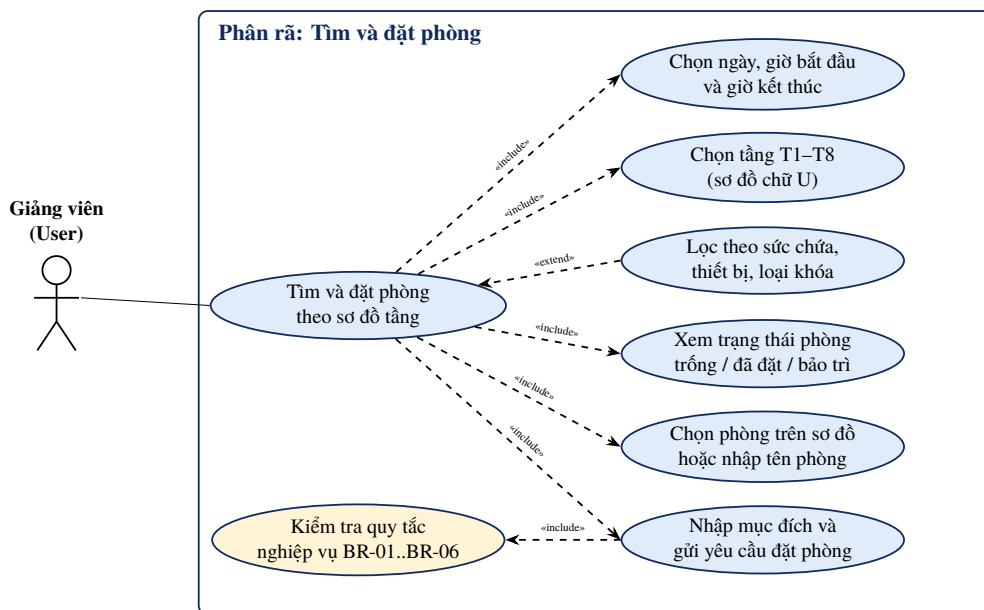
Hình 2.1: Biểu đồ use case tổng quát của hệ thống RoomHub

Các use case dùng chung gồm đăng nhập, đăng ký, khôi phục mật khẩu, xem thông báo và cập nhật hồ sơ. Phía User có bốn use case chính là tìm và đặt phòng, xem lịch đặt phòng (chỉ đọc, phân nhóm sắp tới, đã sử dụng, đã đóng), quản lý đặt

phòng của tôi và hai use case mở rộng là ủy quyền người nhận khóa hộ, tạo mã PIN tạm thời. Phía Admin có bảy use case quản lý, trong đó use case duyệt, từ chối, đổi phòng là trung tâm của nghiệp vụ.

2.2.2 Biểu đồ use case phân rã “Tìm và đặt phòng”

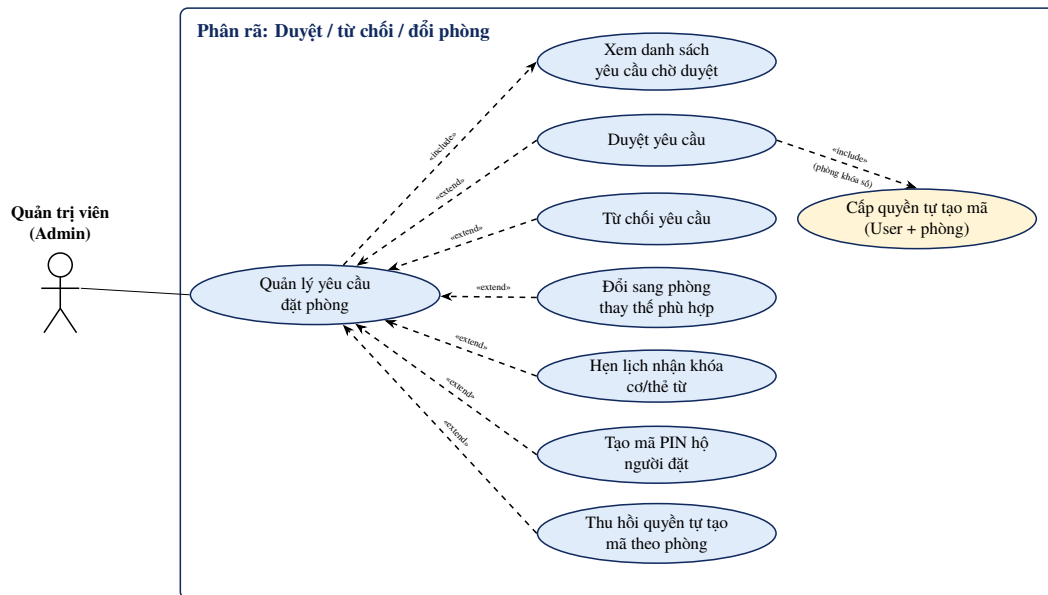
Use case “Tìm và đặt phòng” được phân rã như Hình 2.2. Người dùng bắt buộc chọn khung thời gian, chọn tầng, xem trạng thái phòng trên sơ đồ, chọn phòng và gửi yêu cầu; việc lọc theo sức chứa, thiết bị và loại khóa là tùy chọn. Khi gửi, hệ thống luôn thực hiện use case “Kiểm tra quy tắc nghiệp vụ” với các quy tắc trong Bảng 2.3.



Hình 2.2: Biểu đồ use case phân rã “Tìm và đặt phòng”

2.2.3 Biểu đồ use case phân rã “Duyệt / từ chối / đổi phòng”

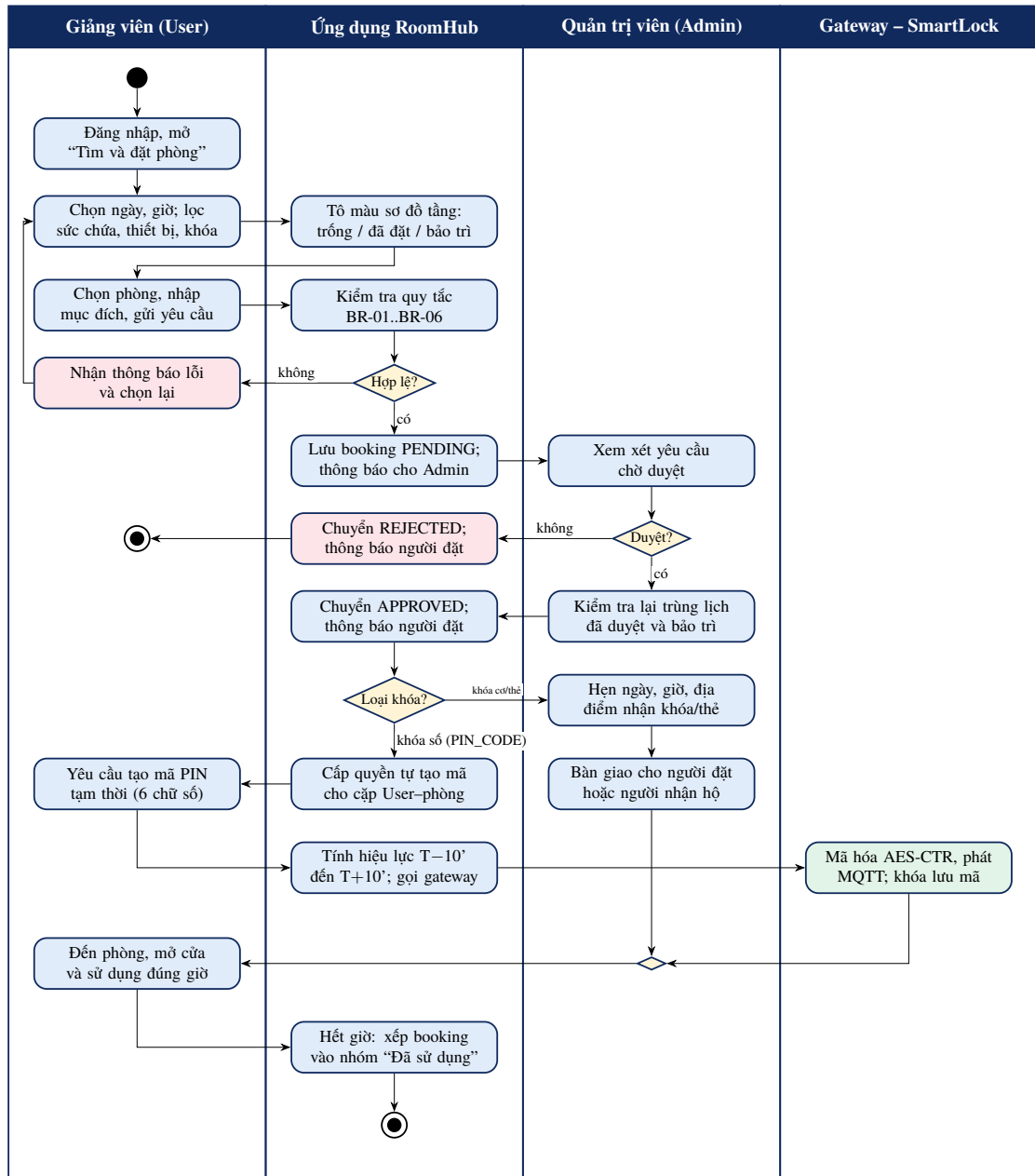
Hình 2.3 phân rã use case quản lý yêu cầu đặt phòng của Admin. Admin xem danh sách yêu cầu chờ duyệt, sau đó có thể duyệt, từ chối, đổi sang phòng thay thế, hẹn lịch nhận khóa cơ/thẻ, tạo mã PIN hộ người đặt hoặc thu hồi quyền tự tạo mã. Khi duyệt yêu cầu tại phòng khóa số, hệ thống tự động cấp quyền tự tạo mã cho đúng cặp người dùng – phòng; quyền này không làm thay đổi vai trò của tài khoản.



Hình 2.3: Biểu đồ use case phân rã “Duyệt / từ chối / đổi phòng”

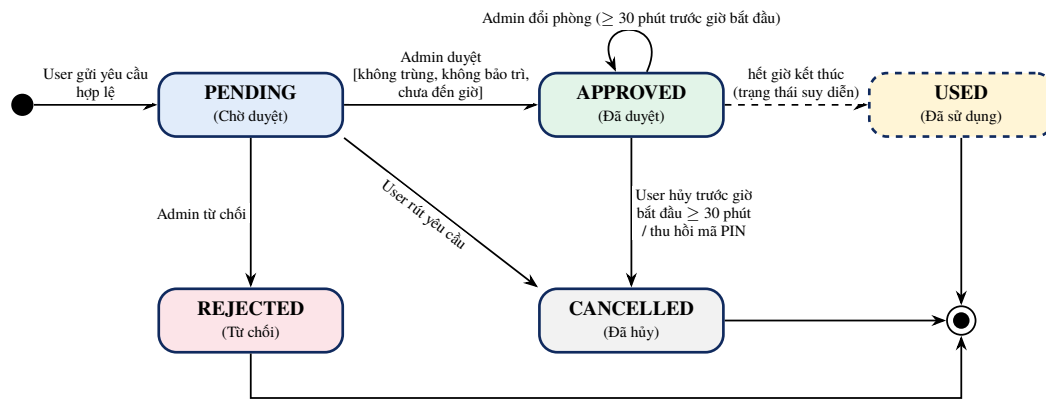
2.2.4 Quy trình nghiệp vụ

Quy trình nghiệp vụ đặt phòng kết hợp nhiều use case của cả hai vai trò, được minh họa bằng biểu đồ hoạt động ở Hình 2.4. Giảng viên chọn khung giờ và bộ lọc, ứng dụng tô màu sơ đồ tầng; giảng viên chọn phòng, nhập mục đích và gửi. Nếu vi phạm quy tắc, ứng dụng hiển thị lý do để người dùng chọn lại. Yêu cầu hợp lệ được lưu với trạng thái chờ duyệt và thông báo tới Admin. Khi Admin duyệt, hệ thống kiểm tra lại xung đột với các booking đã duyệt và lịch bảo trì trước khi chuyển sang trạng thái đã duyệt. Quy trình sau đó rẽ nhánh theo loại khóa: với khóa số, hệ thống cấp quyền và giảng viên tự tạo mã PIN được gửi tới khóa qua gateway; với khóa cơ hoặc thẻ từ, Admin hẹn lịch và bàn giao cho người đặt hoặc người được ủy quyền.

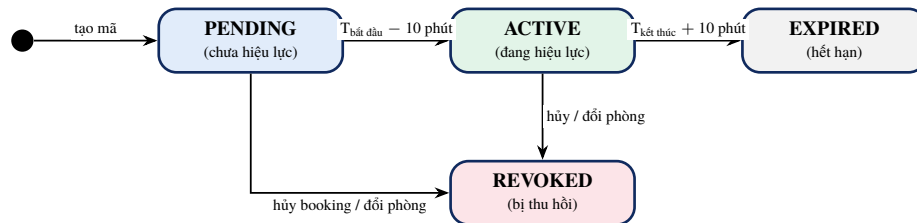


Hình 2.4: Biểu đồ hoạt động của quy trình đặt phòng và cấp quyền tiếp cận

Vòng đời của một booking được mô tả bằng biểu đồ trạng thái ở Hình 2.5. Tài liệu nghiệp vụ [1] đề xuất mười trạng thái (Nháp, Chờ duyệt, Đã duyệt, Chờ bàn giao, Đã nhận thẻ/Đã cấp pass, Đang sử dụng, Hoàn tất, Từ chối, Đã hủy, No-show). Phiên bản hiện tại hiện thực bốn trạng thái lưu trữ là PENDING, APPROVED, REJECTED, CANCELLED; trạng thái “Đã sử dụng” được suy ra khi booking đã duyệt qua giờ kết thúc. Tiến trình tiếp cận không được mã hóa thành trạng thái booking mà gắn với đối tượng riêng là mã PIN tạm thời (có bốn trạng thái hiển thị) hoặc lịch hẹn nhận khóa. Cách tách này phù hợp với khuyến nghị của tài liệu nghiệp vụ là coi phương thức tiếp cận phòng như một phần độc lập của booking.



Vòng đời mã PIN tạm thời gắn với booking APPROVED (hàm getPinDisplayStatus)



Hình 2.5: Biểu đồ trạng thái của booking và của mã PIN tạm thời

2.3 Đặc tả chức năng

Mục này đặc tả năm use case quan trọng nhất. Các use case còn lại gồm hủy đặt phòng, hẹn lịch nhận khóa và ủy quyền, quản lý lịch bảo trì, cấu hình nghiệp vụ và quản lý SmartLock được đặc tả tại Phụ lục A. Trước hết, Bảng 2.3 liệt kê các quy tắc nghiệp vụ mà hệ thống đã hiện thực; các quy tắc được đánh mã để tham chiếu trong đặc tả và kiểm thử.

Bảng 2.3: Các quy tắc nghiệp vụ đã hiện thực trong RoomHub

Mã	Quy tắc	Tham số mặc định
BR-01	Phòng phải tồn tại và ở trạng thái khả dụng (AVAILABLE).	–
BR-02	Ngày có dạng YYYY-MM-DD, giờ có dạng HH:mm và giờ bắt đầu nhỏ hơn giờ kết thúc.	–
BR-03	Chỉ được đặt trước trong khoảng [min, max] ngày tính theo ngày lịch.	1 đến 3 ngày
BR-04	Không được đặt phòng trùng với lịch bảo trì chưa bị hủy của phòng đó.	–
BR-05	Không trùng với yêu cầu PENDING hoặc APPROVED khác của cùng phòng; một User không có hai yêu cầu chồng thời gian.	–
BR-06	Tổng số yêu cầu đang chờ hoặc sắp sử dụng của một User không vượt quá giới hạn; mục đích sử dụng không được để trống.	2 yêu cầu
BR-07	Chỉ duyệt yêu cầu PENDING chưa đến giờ bắt đầu; kiểm tra lại bảo trì và trùng với booking đã duyệt.	–
BR-08	User chỉ hủy booking đã duyệt nếu còn ít nhất một khoảng thời gian trước giờ bắt đầu.	30 phút
BR-09	Admin chỉ đổi phòng trước giờ bắt đầu một khoảng tối thiểu; phòng thay thế phải cùng loại khóa, sức chứa không nhỏ hơn, có đủ thiết bị, không bảo trì và không trùng lịch.	30 phút
BR-10	Mã PIN chỉ tạo cho booking APPROVED tại phòng PIN_CODE đang gắn SmartLock; hiệu lực từ trước giờ bắt đầu đến sau giờ kết thúc một khoảng đệm; bị thu hồi khi hủy hoặc đổi phòng.	10 phút
BR-11	User chỉ tự tạo mã khi có quyền đang hiệu lực cho đúng phòng; Admin có thể thu hồi quyền theo từng phòng.	–
BR-12	Ủy quyền nhận hộ và hèn nhận khóa chỉ áp dụng cho booking đã duyệt tại phòng PHYSICAL_KEY; lịch hèn phải trước giờ sử dụng.	–

2.3.1 Đặc tả use case “Đăng nhập”

Bảng 2.4: Đặc tả use case UC01 – Đăng nhập

Mã use case	UC01
Tác nhân	User, Admin
Mô tả	Người dùng xác thực bằng tên đăng nhập và mật khẩu; hệ thống xác định vai trò và mở giao diện tương ứng.
Tiền điều kiện	Tài khoản đã tồn tại và đang ở trạng thái hoạt động.
Luồng chính	1. Người dùng mở ứng dụng, nhập tài khoản và mật khẩu, bấm “Đăng nhập”. 2. Hệ thống chuẩn hóa tên đăng nhập (bỏ khoảng trắng, chuyển chữ thường). 3. Hệ thống tìm tài khoản khớp mật khẩu và đang hoạt động; ở chế độ máy chủ, dịch vụ Java tạo phiên AuthSession và trả token. 4. Hệ thống mở “Chế độ quản trị viên” nếu vai trò là admin, hoặc “Chế độ người dùng” nếu vai trò là user.
Luồng phát sinh	3a. Sai thông tin hoặc tài khoản bị khóa: hiển thị “Sai tài khoản hoặc mật khẩu” và giữ nguyên màn hình. 3b. Người dùng chọn “Đăng ký” hoặc “Quên mật khẩu?” để tạo tài khoản User mới hoặc đặt lại mật khẩu bằng mã khôi phục.
Hậu điều kiện	Phiên đăng nhập được thiết lập; mọi yêu cầu tiếp theo tới máy chủ mang token Bearer.

2.3.2 Đặc tả use case “Tìm và đặt phòng”

Bảng 2.5: Đặc tả use case UC02 – Tìm và đặt phòng

Mã use case	UC02
Tác nhân	User
Tiền điều kiện	User đã đăng nhập, tài khoản hoạt động.
Luồng chính	1. User mở “Tìm và đặt phòng”; hệ thống sinh danh sách ngày được phép theo cấu hình (mặc định từ ngày mai đến ba ngày sau). 2. User vuốt chọn ngày, giờ bắt đầu, giờ kết thúc và chọn tầng T1–T8. 3. User nhập tùy chọn sức chứa tối thiểu, thiết bị cần có và loại khóa (mọi khóa, mã số, cơ/thẻ). 4. Hệ thống hiển thị bầy phòng của tầng trên sơ đồ chữ U, mỗi ô ghi sức chứa, thiết bị và tô màu trống, đã đặt, bảo trì hoặc không khớp lọc; đồng thời báo số phòng phù hợp đang trống. 5. User chạm chọn một phòng trống (hoặc nhập tên phòng để nhảy tới đúng tầng), nhập mục đích và bấm “Gửi yêu cầu đặt phòng”. 6. Hệ thống kiểm tra BR-01 đến BR-06, lưu booking trạng thái PENDING, tạo thông báo cho Admin và làm mới sơ đồ.
Luồng phát sinh	5a. Phòng được chọn không ở trạng thái trống: hiển thị “Hãy chọn một phòng đang trống và phù hợp bộ lọc”. 6a. Vi phạm một quy tắc: hiển thị thông điệp tương ứng, ví dụ “Phòng có lịch bảo trì trong khoảng thời gian này” và không lưu booking.
Hậu điều kiện	Booking PENDING được lưu; phòng hiển thị màu “đã đặt” trong khung giờ đó; Admin nhận thông báo.

2.3.3 Đặc tả use case “Duyệt yêu cầu đặt phòng”

Bảng 2.6: Đặc tả use case UC03 – Duyệt yêu cầu đặt phòng

Mã use case	UC03
Tác nhân	Admin
Tiền điều kiện	Admin đã đăng nhập; tồn tại booking ở trạng thái PENDING.
Luồng chính	1. Admin mở “Yêu cầu đặt phòng” (hoặc chạm vào thông báo “Có yêu cầu đặt phòng mới”). 2. Hệ thống hiển thị thông tin người đặt, phòng, ngày giờ, mục đích. 3. Admin chọn “Duyệt”. 4. Hệ thống khóa bản ghi booking, kiểm tra BR-07: yêu cầu còn PENDING, chưa đến giờ bắt đầu, không trùng bảo trì, không trùng booking đã duyệt khác. 5. Hệ thống chuyển trạng thái APPROVED, ghi người duyệt và thời điểm duyệt. 6. Nếu phòng dùng khóa mã số, hệ thống cấp quyền tự tạo mã cho cặp User – phòng. 7. Hệ thống gửi thông báo “Yêu cầu đã được duyệt” cho người đặt.
Luồng phát sinh	3a. Admin chọn “Từ chối”: hệ thống chuyển REJECTED và thông báo “Yêu cầu bị từ chối”. 4a. Vi phạm BR-07: hiển thị lý do, trạng thái không đổi.
Hậu điều kiện	Booking ở trạng thái APPROVED hoặc REJECTED; người đặt nhận thông báo; với phòng khóa số, bản ghi PinPermission được kích hoạt.

2.3.4 Đặc tả use case “Đổi phòng thay thế”

Bảng 2.7: Đặc tả use case UC04 – Đổi phòng thay thế

Mã use case	UC04
Tác nhân	Admin
Tiền điều kiện	Booking đã APPROVED và còn ít nhất 30 phút (theo cấu hình) trước giờ bắt đầu.
Luồng chính	1. Admin chọn booking cần đổi, ví dụ khi phòng cũ phải phục vụ sự kiện ưu tiên. 2. Hệ thống chỉ đề xuất phòng thỏa mãn hàm <code>meetsReplacementRoomRequirements</code>: khác phòng cũ, khả dụng, cùng loại khóa, sức chứa không nhỏ hơn và có đủ thiết bị của phòng cũ. 3. Admin chọn phòng thay thế và nhập lý do. 4. Hệ thống kiểm tra BR-09 cùng lịch bảo trì và các yêu cầu chờ/đã duyệt của phòng mới. 5. Hệ thống cập nhật phòng, ghi lịch sử đổi phòng, thu hồi mã PIN cũ và lịch hẹn nhận khóa cũ; nếu phòng mới dùng khóa số thì cấp quyền tạo mã cho phòng mới. 6. Hệ thống thông báo cho người đặt kèm lý do đổi phòng.
Luồng phát sinh	4a. Phòng thay thế khác loại khóa: báo “Phòng thay thế phải có cùng loại khóa với phòng hiện tại”. 4b. Quá hạn đổi phòng hoặc thiếu lý do: báo lỗi tương ứng.
Hậu điều kiện	Booking trở tới phòng mới, danh sách <code>roomChanges</code> có thêm một bản ghi; quyền tiếp cận phòng cũ bị vô hiệu.

2.3.5 Đặc tả use case “Tạo mã PIN tạm thời”

Bảng 2.8: Đặc tả use case UC05 – Tạo mã PIN tạm thời

Mã use case	UC05
Tác nhân	User (chủ booking) hoặc Admin; SmartLock Gateway
Tiền điều kiện	Booking APPROVED tại phòng PIN_CODE; phòng đang được gắn SmartLock; User có quyền tự tạo mã cho phòng; gateway đang hoạt động.
Luồng chính	1. User mở “Đặt phòng của tôi”, chọn booking và bấm tạo mã. 2. Hệ thống kiểm tra BR-10, BR-11 và booking chưa có mã đang hiệu lực. 3. Hệ thống sinh mã sáu chữ số, tính thời điểm hiệu lực từ giờ bắt đầu trừ 10 phút đến giờ kết thúc cộng 10 phút. 4. Hệ thống cấp một passwordId (1–255) của khóa và gửi lệnh tới gateway. 5. Gateway mã hóa mật khẩu, phát trait <code>traitCreateTmp-PasswordLock</code> qua MQTT và trả mã yêu cầu. 6. Hệ thống lưu TemporaryPin vào booking và thông báo “Đã cấp mã mở cửa tạm thời”.
Luồng phát sinh	2a. Quyền đã bị thu hồi: báo “Quyền tự tạo mã tại phòng này đã bị thu hồi hoặc chưa được cấp”. 4a. Phòng chưa gắn khóa: báo “Phòng chưa được gắn với SmartLock đang quản lý”. 5a. Gateway không phản hồi: báo lỗi kết nối, mã không được lưu.
Hậu điều kiện	Khóa lưu mật khẩu tạm thời có thời hạn; người dùng xem mã và trạng thái hiệu lực trong chi tiết booking.

2.4 Yêu cầu phi chức năng

Về **tính đúng đắn và độ tin cậy**, hệ thống không được xác nhận hai booking chồng thời gian trên cùng một phòng, kể cả khi nhiều yêu cầu đến đồng thời; ở máy chủ, bước đọc danh sách booking, kiểm tra và ghi booking mới phải nằm trong cùng một giao dịch có khóa ghi. Mọi thao tác quản trị phải được kiểm tra quyền ở tầng dịch vụ chứ không chỉ dựa vào việc ấn nút trên giao diện.

Về **bảo mật**, các API ngoại trừ đăng nhập yêu cầu token Bearer hợp lệ; token OneIoT không được lưu trong kho mã nguồn; mật khẩu gửi tới khóa phải được mã hóa trước khi phát lên broker; mã PIN chỉ hiển thị cho chủ booking và Admin.

Về **tính khả dụng**, giao diện tiếng Việt, tối ưu cho điện thoại Android màn hình khoảng 6,4 inch; các thao tác chọn ngày giờ dùng cột cuộn dọc để tránh gõ phím; thông điệp lỗi nêu rõ quy tắc bị vi phạm. Ứng dụng phải chạy được độc lập không cần máy tính phát triển (bản Release).

Về **tính dễ bảo trì và mở rộng**, mã nguồn được chia theo module nghiệp vụ, các tham số chính sách được lưu dưới dạng cấu hình thay vì hằng số trong giao

diện, và việc chuyển giữa lưu cục bộ với đồng bộ máy chủ chỉ phụ thuộc một cơ cấu hình.

Về **yêu cầu kỹ thuật**, máy chủ dùng Java 17, Spring Boot 3.4.4, JPA/Hibernate và H2 ở chế độ tệp; ứng dụng dùng React Native 0.86.2, React 19.2.3, TypeScript 5.8; gateway khóa viết bằng Python với thư viện paho-mqtt và cryptography.

Chương này đã xác định ba tác nhân, mười lăm use case, quy trình đặt phòng hai nhánh theo loại khóa, mười hai quy tắc nghiệp vụ và các yêu cầu phi chức năng. Những yêu cầu về kiểm tra quyền phía máy chủ, chống trùng lịch khi truy cập đồng thời và tích hợp khóa thông minh là cơ sở để lựa chọn các công nghệ được trình bày trong Chương 3.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định các yêu cầu chức năng và phi chức năng của RoomHub, trong đó nổi bật là kiểm tra quyền và quy tắc nghiệp vụ ở phía máy chủ, chống trùng lịch khi có truy cập đồng thời, chạy trên điện thoại Android và tích hợp khóa thông minh. Chương này tóm lược nền tảng lý thuyết và công nghệ được dùng để đáp ứng từng nhóm yêu cầu, gồm lập trình hướng đối tượng với Java, khung Spring Boot, JPA và cơ sở dữ liệu H2, kiến trúc REST, React Native với TypeScript, giao thức MQTT cùng mã hóa AES, và các công cụ mô hình hóa, kiểm thử. Với mỗi lựa chọn, báo cáo nêu phương án thay thế và lý do quyết định.

3.1 Lập trình hướng đối tượng với Java 17

Lập trình hướng đối tượng tổ chức chương trình thành các đối tượng gồm dữ liệu và hành vi, dựa trên bốn nguyên lý đóng gói, trừu tượng hóa, kế thừa và đa hình [2]. Java là ngôn ngữ hướng đối tượng có kiểu tĩnh, chạy trên máy ảo JVM, được đặc tả chính thức trong *Java Language Specification* [3]. Phiên bản 17 là bản hỗ trợ dài hạn, bổ sung các cấu trúc giúp mã nghiệp vụ ngắn gọn hơn như `record`, biểu thức `switch` và `var`.

Trong RoomHub, các nguyên lý trên được áp dụng trực tiếp ở dịch vụ `booking-server`. Tính đóng gói thể hiện ở lớp `BookingService`: các phương thức kiểm tra như `validateBooking`, `overlaps`, `hasMaintenanceConflict` được khai báo `private`, chỉ các ca sử dụng như `createBooking`, `reviewBooking` là `public`. Tính kế thừa và đa hình thể hiện ở hai lớp ngoại lệ nghiệp vụ `UnauthorizedException` và `ForbiddenException` cùng kế thừa `RuntimeException`; bộ xử lý ngoại lệ tập trung dựa vào kiểu động của ngoại lệ để chọn mã HTTP 401, 403 hoặc 400. Lớp `SessionAuthFilter` kế thừa lớp trừu tượng `OncePerRequestFilter` và ghi đè phương thức `doFilterInternal`; lớp `EquipmentConverter` hiện thực giao diện `AttributeConverter`. Các đối tượng truyền dữ liệu (DTO) như `BookingInput`, `LoginResult` được khai báo bằng `record` để bất biến và tự sinh `equals`, `hashCode`, đúng khuyến nghị ưu tiên đối tượng bất biến trong [4].

Phương án thay thế là C# trên nền .NET hoặc Node.js với TypeScript ở phía máy chủ. Nhóm chọn Java vì đây là yêu cầu của học phần Lập trình nâng cao, đồng thời hệ sinh thái Java có sẵn khung Spring và đặc tả JPA đã chuẩn hóa, giúp nhóm tập trung vào mô hình nghiệp vụ.

3.2 Spring Boot và cơ chế đảo ngược điều khiển

Spring Framework cung cấp vùng chứa IoC (Inversion of Control) quản lý vòng đời các đối tượng gọi là *bean* và tiêm phụ thuộc (dependency injection) qua hàm khởi tạo. Spring Boot bổ sung cơ chế tự cấu hình, máy chủ web nhúng và các *starter* gom sẵn thư viện [5]. Nhờ đó, một lớp chỉ cần gắn chú thích `@Service`, `@RestController` hoặc `@Component` để được khởi tạo và nối với nhau.

RoomHub sử dụng bốn khả năng của Spring. Thứ nhất, tiêm phụ thuộc qua hàm khởi tạo: `ApiController` nhận `BookingService`, còn `BookingService` nhận `EntityManager`, giúp các lớp không tự tạo phụ thuộc và dễ thay thế khi kiểm thử. Thứ hai, chuỗi bộ lọc Servlet: `SessionAuthFilter` chặn mọi yêu cầu `/api/**` trừ đăng nhập để xác thực token trước khi vào controller, hiện thực mẫu Chain of Responsibility. Thứ ba, xử lý ngoại lệ tập trung bằng `@RestControllerAdvice`, giúp tầng dịch vụ chỉ cần ném ngoại lệ có thông điệp tiếng Việt. Thứ tư, quản lý giao dịch khai báo bằng `@Transactional`: mỗi ca sử dụng ghi dữ liệu chạy trong một giao dịch, các truy vấn chỉ đọc được đánh dấu `readOnly = true`.

Các lựa chọn thay thế gồm Jakarta EE thuần với Servlet hoặc các khung nhẹ như Micronaut, Quarkus. Nhóm chọn Spring Boot 3.4.4 vì tài liệu phong phú, cấu hình tối thiểu và hỗ trợ đầy đủ Java 17.

3.3 Jakarta Persistence, Hibernate và cơ sở dữ liệu H2

Jakarta Persistence (JPA) là đặc tả ánh xạ đối tượng – quan hệ (ORM) của Java, cho phép khai báo lớp thực thể bằng các chú thích `@Entity`, `@Table`, `@Id`, `@Embeddable` và truy vấn bằng ngôn ngữ JPQL [6]. Hibernate là bản hiện thực JPA mặc định trong Spring Boot. Đối tượng `EntityManager` đóng vai trò đơn vị công việc (Unit of Work) và bản đồ định danh (Identity Map) theo cách phân loại của Fowler [7]: thay đổi trên thực thể được theo dõi và ghi xuống cơ sở dữ liệu khi giao dịch xác nhận.

JPA hỗ trợ hai chiến lược điều khiển tương tranh. Khóa lạc quan dùng cột phiên bản và phát hiện xung đột khi ghi; khóa bi quan (`LockModeType.PESSIMISTIC_WRITE`) phát lệnh `SELECT ... FOR UPDATE` để giữ khóa ghi cho tới khi giao dịch kết thúc [8]. Với bài toán đặt phòng, xung đột không nằm trên một bản ghi có sẵn mà nằm ở việc *chèn* một booking mới chồng thời gian, nên RoomHub dùng khóa bi quan trên tập booking trước khi kiểm tra và chèn (xem mục 5.1).

H2 là hệ quản trị cơ sở dữ liệu quan hệ viết bằng Java, chạy nhúng trong tiến trình ứng dụng và hỗ trợ chế độ lưu tệp [9]. RoomHub cấu hình

`jdbc:h2:file:./data/booking` với `ddl-auto=update` để Hibernate tự sinh lược đồ từ các lớp thực thể, kèm giao diện quản trị `/h2-console`. So với MySQL hoặc PostgreSQL, H2 không cần cài đặt máy chủ riêng nên phù hợp giai đoạn phát triển và demo; do mã truy cập dữ liệu chỉ dùng JPA, việc chuyển sang PostgreSQL khi triển khai thật chỉ cần đổi chuỗi kết nối và trình điều khiển.

3.4 Kiến trúc REST và định dạng JSON

REST (Representational State Transfer) là kiểu kiến trúc do Fielding đề xuất, trong đó tài nguyên được định danh bằng URI, thao tác qua các phương thức HTTP chuẩn và giao tiếp không trạng thái giữa máy khách và máy chủ [10]. Dữ liệu trao đổi trong RoomHub dùng JSON do thư viện Jackson của Spring tự chuyển đổi từ các record và thực thể. Tài nguyên chính gồm `/api/rooms`, `/api/bookings`, `/api/maintenance`, `/api/configuration`, `/api/notifications`; các hành động nghiệp vụ được biểu diễn thành tài nguyên con như `POST /api/bookings/{id}/review` và `POST /api/bookings/{id}/temporary-pin`. Xác thực dùng token ngẫu nhiên UUID lưu trong bảng `auth_sessions`, gửi kèm tiêu đề `Authorization: Bearer`.

Các phương án thay thế là GraphQL, phù hợp khi máy khách cần truy vấn linh hoạt nhiều dạng dữ liệu, và gRPC, phù hợp giao tiếp giữa các dịch vụ nội bộ. Với số lượng tài nguyên nhỏ và máy khách di động gọi bằng `fetch` chuẩn, REST là lựa chọn đơn giản và dễ kiểm thử bằng công cụ dòng lệnh.

3.5 React Native và TypeScript

React Native là khung phát triển ứng dụng di động đa nền tảng dựa trên thư viện React; giao diện được mô tả bằng các thành phần khai báo và được hiển thị bằng thành phần gốc của Android hoặc iOS [11]. TypeScript bổ sung hệ thống kiểu tĩnh cho JavaScript, cho phép khai báo kiểu hợp (union type) như `BookingStatus = 'PENDING' | 'APPROVED' | 'REJECTED' | 'CANCELLED'` và phát hiện lỗi truy cập thuộc tính ngay khi biên dịch [12]. Dữ liệu cục bộ được lưu bằng Async Storage, kho khóa – giá trị bất đồng bộ bền vững trên thiết bị [13]. Chức năng thông báo ưu tiên dùng một native module viết bằng Kotlin để tạo kênh thông báo của Android.

Các phương án thay thế gồm Flutter với ngôn ngữ Dart và ứng dụng Android gốc bằng Kotlin. Nhóm chọn React Native với TypeScript vì cùng một mã nguồn có thể build cho Android và iOS, hệ thống kiểu của TypeScript giúp mô hình dữ liệu phía di động tương ứng một – một với các lớp thực thể Java, và môi trường phát triển trùng với môi trường thực hành của nhóm.

3.6 Giao thức MQTT, chuẩn oneM2M và mã hóa AES

MQTT là giao thức nhắn tin theo mô hình xuất bản – đăng ký (publish/subscribe) qua một broker trung gian, có ba mức chất lượng dịch vụ QoS 0, 1, 2 và phần tiêu đề nhỏ, phù hợp thiết bị IoT [14]. Nền tảng OneIoT tổ chức chủ đề (topic) theo ràng buộc MQTT của chuẩn oneM2M, dạng `/oneM2M/req/{originator}/{target}/json`, và gói dữ liệu trong tài nguyên `m2m:cin` [15]. Gateway của RoomHub dùng thư viện `paho-mqtt`, kết nối broker bằng MQTT 3.1.1 qua TLS tại cổng 2111 với QoS 1, bảo đảm bản tin tạo mật khẩu được chuyển ít nhất một lần.

Mật khẩu tạm thời không được gửi dạng rõ mà mã hóa bằng AES ở chế độ CTR. Chế độ CTR biến mã khối thành mã dòng bằng cách mã hóa một bộ đếm rồi XOR với bản rõ, nên độ dài bản mã bằng độ dài bản rõ và không cần đệm [16]. Khóa 256 bit được dẫn xuất bằng SHA-256 từ định danh thiết bị, còn giá trị khởi tạo được dẫn xuất từ khoảng hiệu lực và số hiệu mật khẩu theo định dạng mà firmware khóa DLWA12 yêu cầu.

Phương án thay thế là điều khiển khóa qua Bluetooth Low Energy khi người dùng đứng trước cửa, hoặc để máy chủ Java gọi trực tiếp broker. Nhóm chọn tách gateway thành tiến trình riêng vì chỉ gateway giữ token OneIoT, ứng dụng di động chỉ gọi một API HTTP đơn giản, và loại khóa có thể thay đổi mà không ảnh hưởng dịch vụ đặt phòng.

3.7 Mô hình hóa UML và kiểm thử tự động

Các biểu đồ use case, hoạt động, trạng thái, gói, lớp và trình tự trong báo cáo tuân theo đặc tả UML 2.5.1 [17]. Kiểm thử tự động phía ứng dụng dùng Jest với preset của React Native [18]; các ca kiểm thử gọi trực tiếp tầng dịch vụ trên kho Async Storage giả lập, nhờ đó kiểm tra được quy tắc nghiệp vụ mà không cần giao diện. Chất lượng mã được kiểm soát thêm bằng ESLint và trình biên dịch TypeScript ở chế độ `-noEmit`.

Bảng 3.1 tổng hợp liên hệ giữa yêu cầu, công nghệ được chọn và phương án thay thế đã cân nhắc.

Bảng 3.1: Tổng hợp lựa chọn công nghệ theo yêu cầu

Yêu cầu (Chương 2)	Công nghệ chọn	Phương án thay thế
Mô hình nghiệp vụ hướng đối tượng, kiểm tra quyền phía máy chủ	Java 17, Spring Boot 3.4.4	C#/.NET, Node.js, Jakarta EE thuần
Lưu trữ, chống trùng lịch đồng thời	JPA/Hibernate, khóa bi quan, H2	JDBC thủ công, MyBatis; PostgreSQL, MySQL
Giao tiếp ứng dụng – máy chủ	REST/JSON, token Bearer	GraphQL, gRPC
Ứng dụng trên điện thoại Android	React Native 0.86.2, TypeScript 5.8	Flutter, Kotlin gốc
Lưu cục bộ khi chưa có máy chủ	Async Storage	SQLite, MMKV
Gửi mật khẩu tạm thời tới khóa	Gateway Python, MQTT/oneM2M, AES-256-CTR	BLE trực tiếp, máy chủ Java gọi broker
Kiểm thử quy tắc nghiệp vụ	Jest, ESLint, TypeScript compiler	Kiểm thử thủ công

Chương này đã trình bày cơ sở lý thuyết và lý do lựa chọn từng công nghệ. Java cùng Spring Boot và JPA đáp ứng yêu cầu mô hình hóa nghiệp vụ và kiểm soát giao dịch ở máy chủ; React Native với TypeScript đáp ứng yêu cầu chạy trên điện thoại; MQTT và AES-CTR phục vụ tích hợp khóa thông minh. Chương 4 sẽ trình bày cách các công nghệ này được tổ chức thành kiến trúc và thiết kế chi tiết của hệ thống.

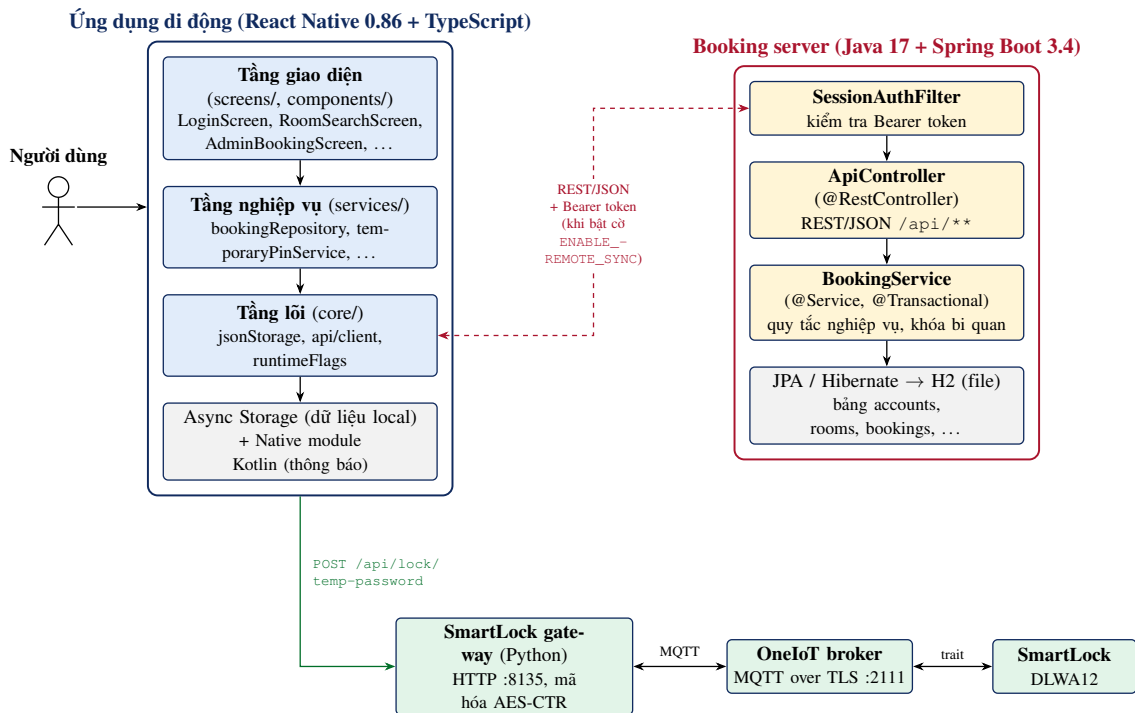
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã trình bày các công nghệ được lựa chọn. Chương này mô tả cách tổ chức các công nghệ đó thành hệ thống RoomHub, đi từ thiết kế kiến trúc, thiết kế gói, thiết kế giao diện, thiết kế lớp và cơ sở dữ liệu tới kết quả xây dựng, kiểm thử và triển khai trên thiết bị thật.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

RoomHub áp dụng kiến trúc khách – chủ nhiều tầng. Ở mức hệ thống có ba thành phần triển khai độc lập như Hình 4.1: ứng dụng di động React Native, dịch vụ nghiệp vụ Java booking-server và gateway khóa thông minh. Ứng dụng giao tiếp với máy chủ qua REST/JSON khi bật cờ ENABLE_REMOTE_SYNC, và giao tiếp với gateway qua HTTP để gửi mật khẩu tạm thời; gateway tiếp tục chuyển lệnh tới khóa qua broker MQTT của OneIoT.



Hình 4.1: Kiến trúc tổng thể của hệ thống RoomHub

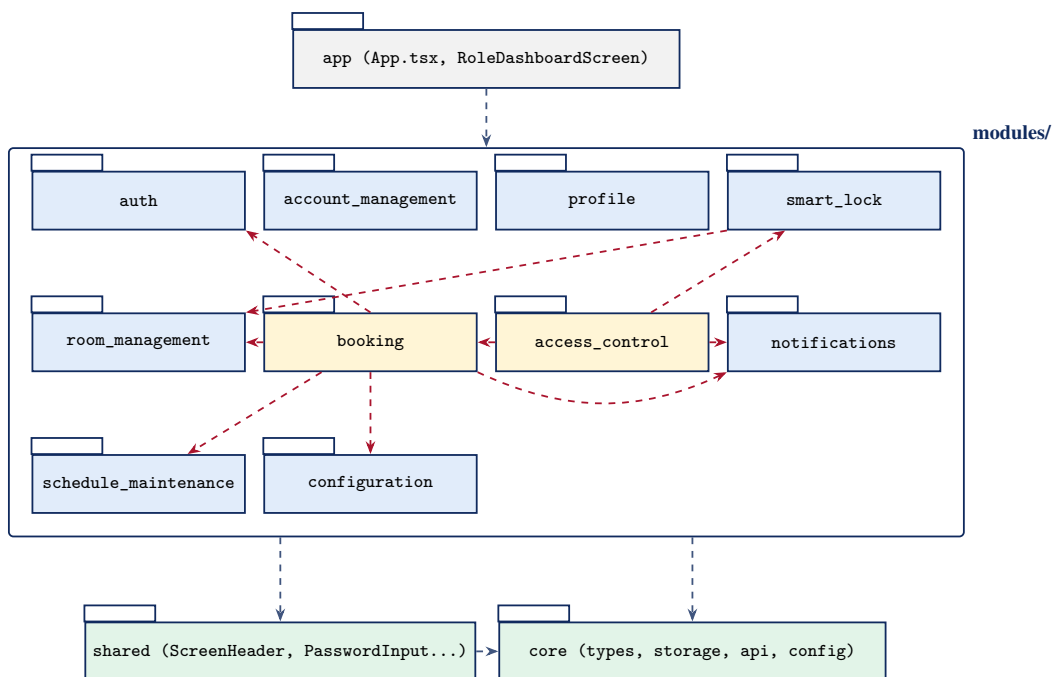
Bên trong mỗi thành phần, nhóm áp dụng kiến trúc phân tầng theo nguyên tắc tầng trên chỉ phụ thuộc tầng dưới [19]. Ở dịch vụ Java, tầng trình diễn API gồm SessionAuthFilter và ApiController; tầng nghiệp vụ là BookingService theo mẫu Service Layer [7], nơi tập trung mọi quy tắc BR-01 đến

BR-11; tầng dữ liệu gồm các lớp thực thể trong lớp bao Domain và EntityManager của JPA. Controller không chứa quy tắc nghiệp vụ mà chỉ lấy tài khoản đã xác thực từ thuộc tính yêu cầu rồi ủy quyền cho dịch vụ; lỗi nghiệp vụ được chuyển thành mã HTTP tại ApiExceptionHandler.

Ở ứng dụng di động, mỗi module nghiệp vụ tự chia ba tầng: screens/ và components/ là tầng giao diện, services/ là tầng nghiệp vụ theo mẫu Repository, model/ chứa kiểu dữ liệu. Tầng core/ cung cấp hạ tầng dùng chung gồm kho JSON trên Async Storage, máy khách API và cờ cấu hình. Mỗi hàm repository kiểm tra isRemoteApiEnabled(): nếu bật thì gọi REST tới máy chủ Java, nếu tắt thì thực thi cùng bộ quy tắc trên dữ liệu cục bộ. Nhờ vậy giao diện không cần biết dữ liệu đến từ đâu; chi tiết giải pháp được trình bày tại mục 5.3.

4.1.2 Thiết kế tổng quan

Hình 4.2 là biểu đồ gói của ứng dụng di động. Gói app ghép luồng màn hình theo vai trò và chỉ phụ thuộc các module. Mười module nghiệp vụ nằm trong modules/, cùng phụ thuộc hai gói nền shared (thành phần giao diện dùng lại như ScreenHeader, PasswordInput) và core (kiểu UserRole, AuthenticatedUser, lưu trữ, API). Các mũi tên màu đỏ thể hiện phụ thuộc chính giữa các module: booking dùng room_management, schedule_maintenance, configuration, access_control, notifications và smart_lock; access_control dùng booking và notifications; notifications dùng access_control; smart_lock dùng booking; booking dùng room_management, schedule_maintenance, configuration, access_control và notifications.



Hình 4.2: Biểu đồ gói của ứng dụng di động RoomHub

Giữa `booking` và `access_control` tồn tại phụ thuộc hai chiều: duyệt `booking` cần cấp quyền PIN, còn tạo PIN cần đọc `booking`. Nhóm hạn chế ảnh hưởng của vòng phụ thuộc này bằng quy ước chỉ import trực tiếp tệp repository cụ thể (`roomPinPermissionRepository`) thay vì import qua `index.ts` của module, để hai module không nạp phần giao diện của nhau. Bảng 4.1 tóm tắt nhiệm vụ của từng module.

Bảng 4.1: Nhiệm vụ các module nghiệp vụ của ứng dụng di động

Module	Nhiệm vụ	Vai trò
auth	Đăng ký, đăng nhập, đăng xuất, đổi và khôi phục mật khẩu, kiểm tra vai trò	Chung
account_management	Thêm, khóa/mở, đặt lại mật khẩu, xóa tài khoản; thu hồi quyền tạo mã theo phòng	Admin
room_management	CRUD phòng, tầng, sức chứa, thiết bị, loại khóa, trạng thái; màn tìm và đặt phòng	Admin, User
booking	Tạo, duyệt, từ chối, hủy, đổi phòng; hẹn nhận khóa; ủy quyền; lịch đặt phòng	Admin, User
access_control	Quyền tạo mã theo cặp User – phòng; sinh và tính hiệu lực mã PIN	Admin, User
smart_lock	Gắn/tháo SmartLock với phòng, cấu hình gateway, cấp passwordId, gửi lệnh	Admin
schedule_maintenance	Lịch sử dụng phòng; tạo/hủy lịch bảo trì; kiểm tra xung đột bảo trì	Admin
configuration	Tham số đặt trước, giới hạn yêu cầu, hạn hủy/đổi, khoảng đệm mã, bật/tắt thông báo	Admin
notifications	Danh sách thông báo, đếm chưa đọc, banner ưu tiên và thông báo hệ thống Android	Chung
profile	Xem, sửa họ tên, email, số điện thoại, đơn vị	Chung

4.1.3 Thiết kế chi tiết gói

Dịch vụ Java được tổ chức trong gói `com.bookingclassroom.server` gồm bảy tệp nguồn. Mặc dù nằm chung một gói, các lớp được phân vai rõ ràng theo tầng như Hình 4.4: `BookingServerApplication` là điểm khởi động; `SessionAuthFilter`, `ApiController`, `ApiExceptionHandler` thuộc tầng API; `BookingService` thuộc tầng nghiệp vụ; lớp `Domain` gom tám lớp thực thể lỏng tĩnh và một bộ chuyển đổi thuộc tính; `DataSeeder` hiện thực `CommandLineRunner` để khởi tạo dữ liệu mẫu gồm hai tài khoản, 56 phòng trên tám tầng, cấu hình mặc định và lịch bảo trì phòng A103 trong ba ngày tới.

Các quan hệ giữa lớp gồm: kết hợp một – một từ `SessionAuthFilter` và `ApiController` tới `BookingService`; phụ thuộc từ dịch vụ tới các record DTO và ngoại lệ; kế thừa từ hai ngoại lệ nghiệp vụ tới `RuntimeException`; hợp

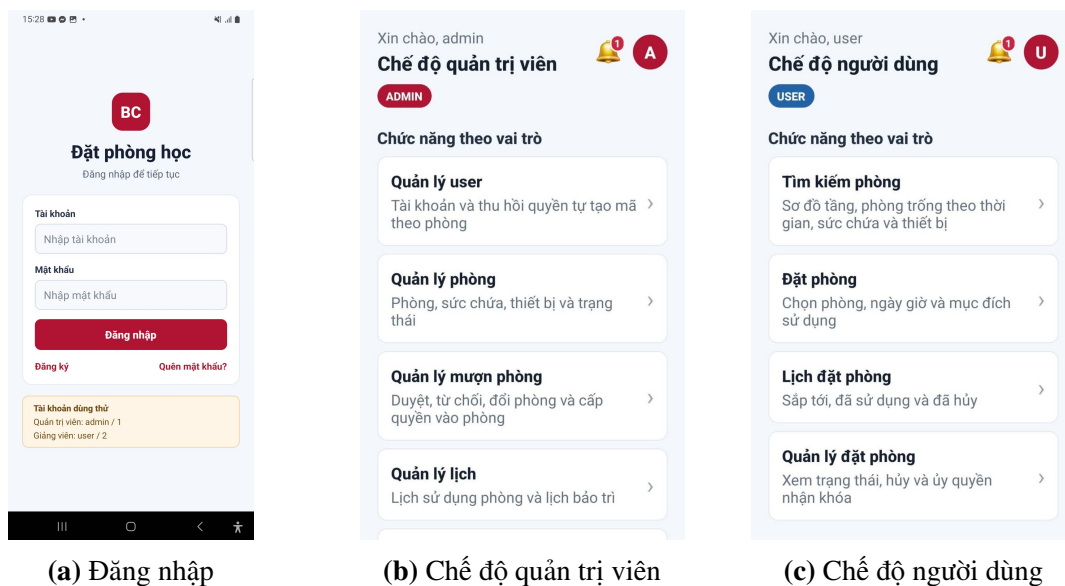
thành giữa Booking và TemporaryPin do mã PIN được nhúng (@Embedded) và không tồn tại độc lập với booking. Các liên kết giữa thực thể (booking – phòng, booking – tài khoản) được lưu bằng khóa ngoại logic dạng chuỗi để đồng bộ trực tiếp với mô hình JSON của ứng dụng di động.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Ứng dụng hướng tới điện thoại Android từ phiên bản 7.0 (API 24) đến Android 16 (API 36), kiểm thử chính trên Samsung M21 màn hình 6,4 inch độ phân giải 1080×2340 điểm ảnh, hiển thị theo chiều dọc. Giao diện thống nhất các quy ước sau: màu chủ đạo đỏ #B01432 gọi màu nhận diện của trường cho nút hành động chính và nút quay lại; chữ tiêu đề màu xanh đậm #172033; nền trang #F4F7FB; mỗi màn hình có ScreenHeader với nút quay lại bên trái và tiêu đề ở giữa. Trạng thái phòng được mã hóa màu nhất quán: xanh nhạt #DDF4E7 là trống, hồng #FBE1E4 là đã đặt, xám #ECEE2 là bảo trì, trắng là không khớp bộ lọc. Thông điệp phản hồi được đặt ngay phía trên nút gửi, màu đỏ khi lỗi và màu xanh khi thành công; các thông báo quan trọng như “Có yêu cầu đặt phòng mới” còn hiển thị thành banner ở đầu màn hình và thông báo hệ thống của Android.

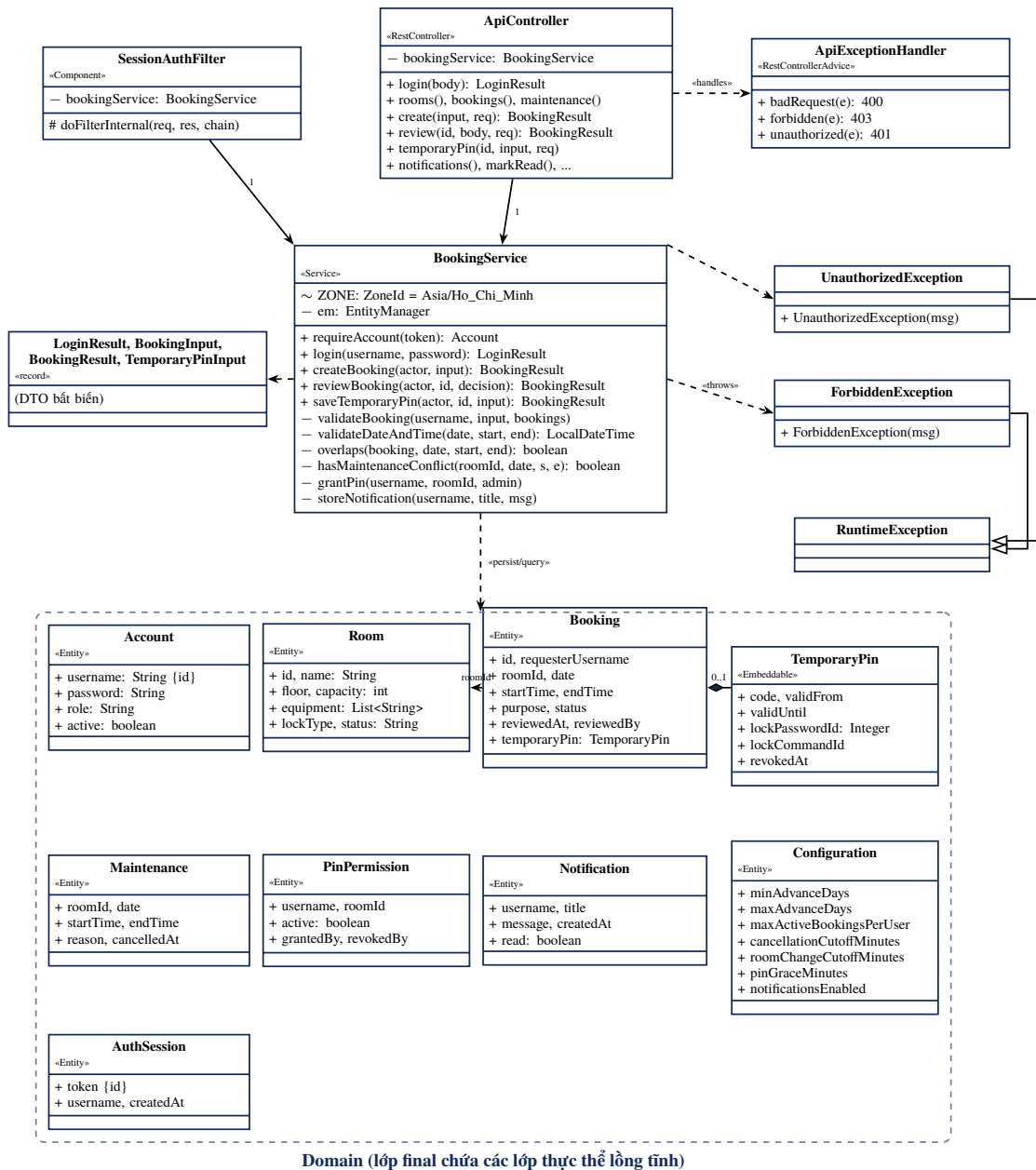
Sau khi đăng nhập, người dùng vào bảng điều khiển theo vai trò như Hình 4.3. Góc phải có chuông thông báo kèm số chưa đọc và ảnh đại diện mở menu hồ sơ, đổi mật khẩu, đăng xuất. Phần thân là danh sách thẻ chức năng: sáu thẻ quản lý cho Admin và ba thẻ dịch vụ cho User.



Hình 4.3: Màn hình đăng nhập và bảng điều khiển theo vai trò

4.2.2 Thiết kế lớp

Hình 4.4 là biểu đồ lớp của dịch vụ Java. Hai lớp chủ đạo được thiết kế chi tiết là BookingService và Booking.



Hình 4.4: Biểu đồ lớp của dịch vụ nghiệp vụ Java booking-server

Lớp BookingService (Bảng 4.2) giữ một tham chiếu EntityManager được tiêm qua hàm khởi tạo và hằng ZONE là múi giờ Asia/Ho_Chi_Minh để mọi phép so sánh thời gian nhất quán dù máy chủ đặt ở đâu. Các phương thức công khai tương ứng các ca sử dụng; các phương thức riêng tư đóng gói từng quy tắc để có thể dùng lại giữa bước tạo và bước duyệt.

Bảng 4.2: Thiết kế chi tiết lớp BookingService

Thành phần	Ý nghĩa
-em: EntityManager	Truy cập dữ liệu JPA, khóa bản ghi
~ZONE: ZoneId	Múi giờ nghiệp vụ Asia/Ho_Chi_Minh
+requireAccount(token): Account	Tra phiên, trả tài khoản hoạt động hoặc ném UnauthorizedException
+login(username, password)	Chuẩn hóa tên, kiểm tra mật khẩu, tạo Auth-Session với token UUID
+createBooking(actor, input)	Kiểm tra vai trò, khóa bí quan, validate-Booking, lưu PENDING, thông báo Admin
+reviewBooking(actor, id, decision)	Khóa booking, kiểm tra BR-07, cập nhật trạng thái, grantPin nếu phòng khóa số
+saveTemporaryPin(actor, id, input)	Kiểm tra quyền chủ booking/Admin, định dạng mã, passwordId, tính khoảng hiệu lực
-validateBooking(username, input, list)	Kiểm tra tuần tự BR-01 đến BR-06
-overlaps(booking, date, start, end)	Phép giao hai khoảng thời gian nửa mở
-hasMaintenanceConflict(room, date, s, e)	Tìm lịch bảo trì chưa hủy giao với khoảng yêu cầu
-grantPin(username, roomId, admin)	Tạo mới hoặc kích hoạt lại PinPermission (upsert)

Lớp thực thể Booking gồm định danh, người đặt, phòng, ngày, giờ bắt đầu, giờ kết thúc, mục đích, trạng thái, thời điểm tạo, người và thời điểm duyệt. Đối tượng TemporaryPin được nhúng với các cột có tiền tố temporary_pin_ nhờ @AttributeOverrides, như Đoạn mã 4.1. Ngày và giờ được lưu dạng chuỗi ISO (YYYY-MM-DD, HH:mm) để so sánh từ điển cho kết quả trùng với so sánh thời gian và để khớp định dạng JSON của ứng dụng.

```

1 @Entity(name = "Booking")
2 @Table(name = "bookings")
3 public static class Booking {
4     @Id public String id;
5     public String requesterUsername;
6     public String roomId;
7     public String date;           // YYYY-MM-DD
8     public String startTime;      // HH:mm
9     public String endTime;
10    @Column(length = 1000) public String purpose;
11    public String status;         // PENDING|APPROVED|REJECTED|CANCELLED
12    public String reviewedAt, reviewedBy;
13    @Embedded
14    @AttributeOverrides({

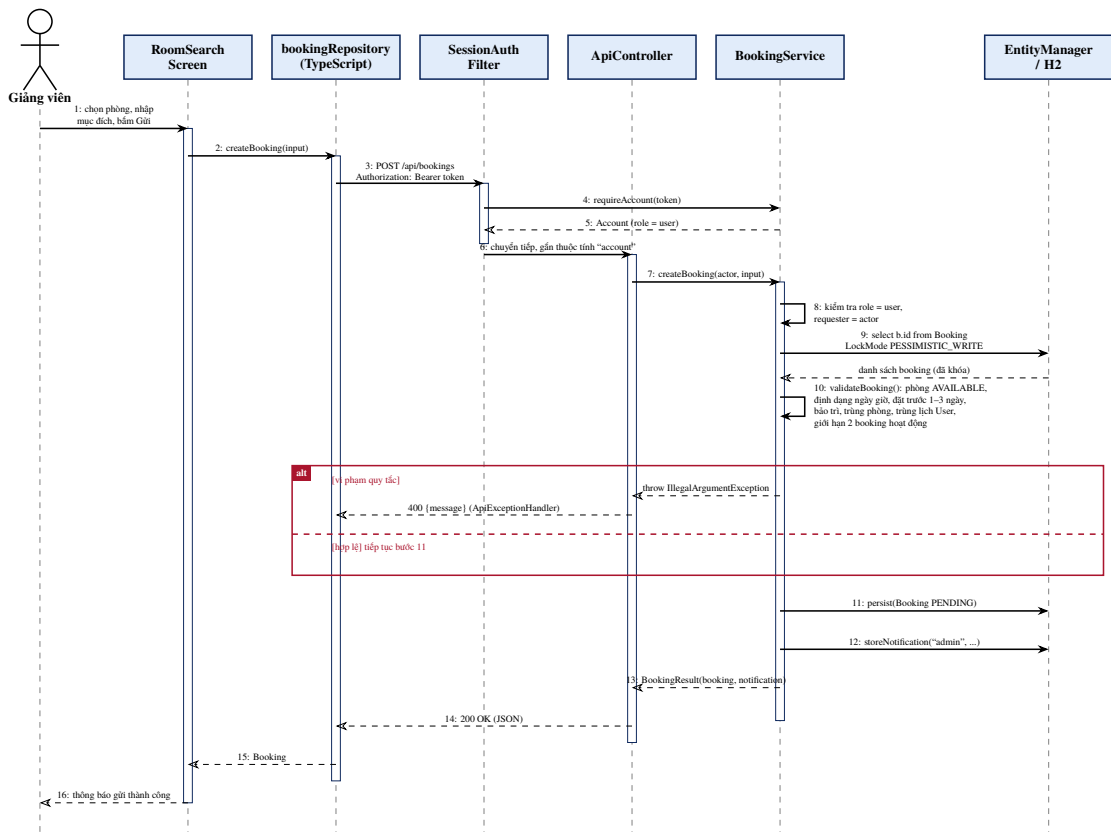
```

```

15     @AttributeOverride(name = "code",
16         column = @Column(name = "temporary_pin_code")),
17     @AttributeOverride(name = "validFrom",
18         column = @Column(name = "temporary_pin_valid_from")),
19     // ... validUntil, lockPasswordId, lockCommandId, revokedAt
20 })
21 public TemporaryPin temporaryPin;
22 }
    
```

Đoạn mã 4.1: Khai báo thực thể Booking với TemporaryPin nhúng (rút gọn)

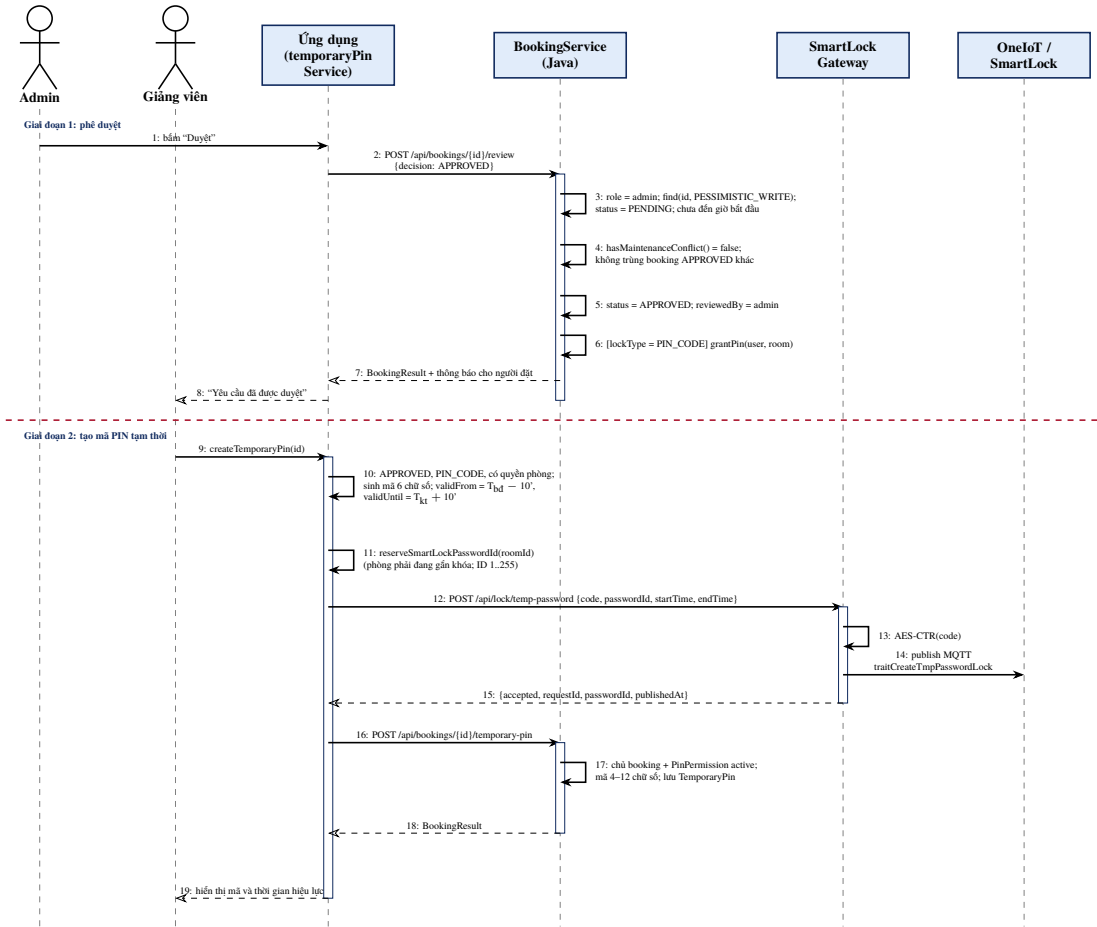
Luồng thông điệp giữa các đối tượng cho hai ca sử dụng quan trọng được thể hiện bằng biểu đồ trình tự. Hình 4.5 mô tả ca tạo yêu cầu đặt phòng qua máy chủ: yêu cầu đi qua bộ lọc xác thực, controller, rồi tới BookingService, nơi tập booking được khóa ghi trước khi kiểm tra và chèn. Nếu vi phạm quy tắc, ngoại lệ `IllegalArgumentException` được `ApiExceptionHandler` chuyển thành phản hồi 400 kèm thông điệp.



Hình 4.5: Biểu đồ trình tự ca sử dụng “Tìm và đặt phòng” qua dịch vụ Java

Hình 4.6 mô tả hai giai đoạn duyệt yêu cầu và tạo mã PIN. Ở giai đoạn duyệt, dịch vụ khóa bản ghi booking bằng `find(..., PESSIMISTIC_WRITE)` để hai quản trị viên không thể duyệt đồng thời, sau đó cấp quyền theo phòng. Ở giai

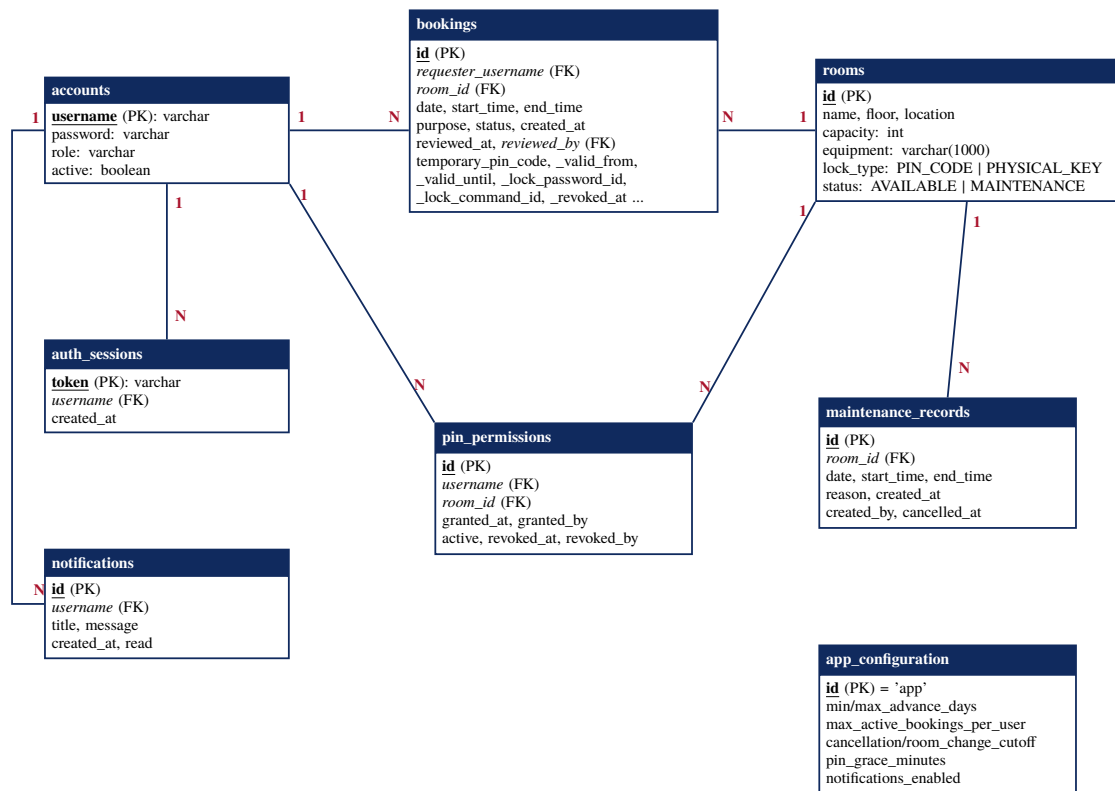
đoạn tạo mã, ứng dụng xin một passwordId của khóa, gọi gateway, rồi mới lưu mã vào booking; nhờ thứ tự này, mã chỉ được ghi nhận khi lệnh đã được phát tới khóa.



Hình 4.6: Biểu đồ trình tự ca sử dụng “Duyệt yêu cầu” và “Tạo mã PIN tạm thời”

4.2.3 Thiết kế cơ sở dữ liệu

Từ các lớp thực thể, Hibernate sinh tám bảng trong H2. Hình 4.7 là biểu đồ thực thể liên kết. Bảng `accounts` liên kết một – nhiều với `auth_sessions`, `notifications`, `bookings` và `pin_permissions`; bảng `rooms` liên kết một – nhiều với `bookings`, `maintenance_records` và `pin_permissions`. Bảng `pin_permissions` thực chất là quan hệ nhiều – nhiều giữa tài khoản và phòng có thêm thuộc tính trạng thái và lịch sử cấp/thu hồi. Bảng `app_configuration` chỉ có một dòng với khóa 'app'.



Hình 4.7: Biểu đồ thực thể liên kết của cơ sở dữ liệu RoomHub

Danh sách thiết bị của phòng được lưu trong một cột `equipment` độ dài 1000 ký tự nhờ `EquipmentConverter` nối các phần tử bằng ký tự xuống dòng. Cách này đơn giản hóa lược đồ cho phiên bản hiện tại; khi cần quản lý số lượng và tình trạng từng thiết bị theo quy tắc BR-13 đến BR-16 của tài liệu nghiệp vụ, cột này sẽ được tách thành hai bảng `equipment` và `room_equipment`. Bảng 4.3 mô tả các cột của bảng trung tâm `bookings`.

Bảng 4.3: Cấu trúc bảng bookings

Cột	Kiểu	Mô tả
id	varchar (PK)	Định danh dạng booking- <UUID>
requester_username	varchar	Tài khoản người đặt
room_id	varchar	Phòng được đặt
date, start_time, end_time	varchar	Ngày và khoảng giờ sử dụng
purpose	varchar(1000)	Mục đích sử dụng
status	varchar	PENDING, APPROVED, RE- JECTED, CANCELLED
created_at, reviewed_at	varchar	Thời điểm tạo, duyệt (ISO-8601)
reviewed_by	varchar	Admin xử lý yêu cầu
temporary_pin_code	varchar	Mã PIN tạm thời
temporary_pin_valid_from/_ - until	varchar	Khoảng hiệu lực đã cộng đệm
temporary_pin_lock_pass- word_id	integer	Vị trí mật khẩu trên khóa (1–255)
temporary_pin_lock_com- mand_id	varchar	Mã yêu cầu gateway trả về
temporary_pin_revoked_at	varchar	Thời điểm thu hồi mã

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.4 liệt kê các ngôn ngữ, thư viện và công cụ cùng phiên bản đã dùng.

Bảng 4.4: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ / thư viện	Địa chỉ URL
Ngôn ngữ máy chủ	Java (OpenJDK) 17	https://openjdk.org
Khung ứng dụng máy chủ	Spring Boot 3.4.4 (web, data-jpa)	https://spring.io
Cơ sở dữ liệu	H2 Database (chế độ tệp)	https://h2database.com
Quản lý build Java	Apache Maven	https://maven.apache.org
Khung ứng dụng di động	React Native 0.86.2, React 19.2.3	https://reactnative.dev
Ngôn ngữ di động	TypeScript 5.8, Kotlin 2.1.20	https://www.typescriptlang.org
Lưu trữ cục bộ	Async Storage 3.1.1	https://react-native-async-storage.github.io
Build Android	Android SDK 36, Gradle, Node.js 22	https://developer.android.com
Kiểm thử, kiểm tra mã	Jest 29, ESLint 8, Prettier 2.8	https://jestjs.io
Gateway khóa	Python 3, paho-mqtt 2.x, cryptography	https://pypi.org/project/paho-mqtt
Quản lý mã nguồn	Git, GitHub	https://github.com

4.3.2 Kết quả đạt được

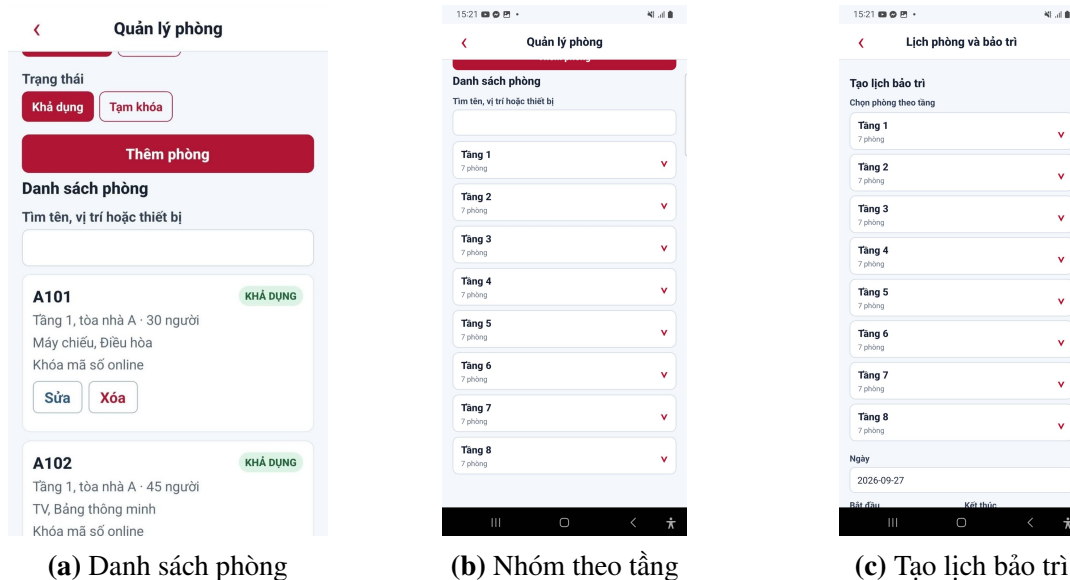
Sản phẩm của bài tập lớn gồm bốn thành phần được lưu trong kho mã nguồn [20]: (i) ứng dụng di động `booking_classroom` kèm tệp cài đặt `booking_classroom-v1.0.apk` chạy độc lập, không cần Metro hay cáp USB; (ii) dịch vụ Java `booking-server` cung cấp REST API, lưu dữ liệu vào H2 và tự khởi tạo dữ liệu mẫu; (iii) gateway `tunghv3` nhận lệnh HTTP và phát bản tin MQTT tới SmartLock; (iv) bộ tài liệu đề xuất dự án, sơ đồ workflow và biên bản hiện trạng triển khai. Hai tài khoản mẫu là `admin/1` và `user/2`. Bảng 4.5 thống kê quy mô mã nguồn.

Chỉ số	Giá trị
Số tệp TypeScript/TSX trong <code>src/</code> của ứng dụng	61 tệp
Số dòng mã TypeScript/TSX (<code>src/</code> và <code>App.tsx</code>)	5.241 dòng
Số module nghiệp vụ / số màn hình	10 module / 15 màn hình
Số dòng mã kiểm thử Jest (4 tệp, 14 ca)	289 dòng
Số dòng mã Kotlin (native module thông báo, Activity)	197 dòng
Số lớp Java (tính cả lớp lồng, record) / số dòng	23 lớp / 860 dòng
Số lớp thực thể JPA / số bảng	8 / 8
Số điểm cuối REST API	12
Số dòng mã gateway Python	399 dòng
Dung lượng APK Release v1.0	57.942.770 byte ($\approx$ 55,3 MB)
Dữ liệu mẫu	8 tầng $\times$ 7 phòng = 56 phòng

Tìm và đặt phòng trên sơ đồ tầng. Hình 4.8 là chức năng quan trọng nhất của User. Phần trên cho phép vuốt dọc ba cột ngày, giờ bắt đầu, giờ kết thúc; dòng tóm tắt hiển thị khoảng đã chọn, ví dụ 2026-09-26 08:00–09:00. Bộ lọc gồm sức chứa tối thiểu, thiết bị cần có và loại khóa. Phần dưới là cột chọn tầng T1–T8 và sơ đồ chữ U của tầng đang chọn: mỗi ô phòng ghi tên, số chỗ, thiết bị và được tô màu theo trạng thái trong khung giờ. Dòng “Tầng 1: 7/7 phòng phù hợp đang trống” giúp người dùng biết nhanh có nên chuyển tầng hay không. Khi chọn một phòng trống, ô nhập mục đích và nút gửi xuất hiện ngay trên cùng màn hình.

(a) Chọn khung giờ, bộ lọc và sơ đồ **(b)** Nhập phòng, mục đích và gửi

Quản lý phòng và lịch bảo trì. Hình 4.9 minh họa các màn hình của Admin. Danh sách 56 phòng được gom thành tám thanh tầng có thể mở rộng để tránh cuộn dài; mỗi phòng hiển thị vị trí, sức chứa, thiết bị, loại khóa và nhãn trạng thái *Khả dụng* hoặc *Tạm khóa*, kèm nút Sửa, Xóa. Màn lịch bảo trì dùng cùng bộ chọn theo tầng để tạo lịch bảo trì theo ngày và khung giờ; hệ thống chặn tạo bảo trì trùng booking đã duyệt.



Hình 4.9: Màn hình quản lý phòng và lịch bảo trì của Admin

Quản lý tài khoản, quyền theo phòng và cấu hình nghiệp vụ. Hình 4.10 gồm hai màn hình quản trị. Màn “Quản lý user và quyền phòng” cho phép thêm tài khoản User với mật khẩu ban đầu và mã khôi phục, tìm theo tên đăng nhập, khóa/mở, đặt lại mật khẩu, xóa và xem danh sách phòng mà User được tự tạo mật khẩu tạm thời. Dòng chú thích nhấn mạnh quyền được cấp tự động khi duyệt booking phòng khóa số và việc thu hồi không đổi User thành Admin. Màn “Cấu hình nghiệp vụ” cho phép chỉnh số ngày đặt trước tối thiểu, tối đa, số booking hoạt động tối đa mỗi User, hạn hủy, hạn đổi phòng và khoảng đệm hiệu lực mã số.

Quản lý user và quyền phòng

Thêm tài khoản User

Tên đăng nhập

Mật khẩu ban đầu

Mã khôi phục

Thêm tài khoản

Tài khoản và quyền tự tạo mã theo phòng

Quyền được cấp tự động khi duyệt booking phòng khóa số. Thu hồi tại đây không đổi User thành Admin toàn hệ thống.

Tìm theo tên đăng nhập

@user USER

Phòng được tự tạo mật khẩu tạm thời
Chưa có quyền ở phòng nào.

Khóa Đặt MK: 1234 Xóa

Cấu hình nghiệp vụ

Đặt trước tối thiểu (ngày)

1

Đặt trước tối đa (ngày)

3

Booking hoạt động tối đa / User

2

Hạn hủy booking đã duyệt (phút trước giờ bắt đầu)

30

Hạn Admin đổi phòng (phút trước giờ bắt đầu)

30

Khoảng đệm hiệu lực mã số (phút)

10

(a) Quản lý user và quyền phòng

(b) Cấu hình nghiệp vụ

Hình 4.10: Màn hình quản lý quyền theo phòng và cấu hình tham số nghiệp vụ

4.4 Kiểm thử

Nhóm kết hợp hai mức kiểm thử. Mức thứ nhất là kiểm thử tự động bằng Jest trên tầng dịch vụ của ứng dụng, áp dụng kỹ thuật hộp đen với phân hoạch tương đương và phân tích giá trị biên: mỗi quy tắc được thử với một đầu vào hợp lệ và các đầu vào vi phạm tiêu biểu, ví dụ đặt trước 1 ngày (biên dưới hợp lệ) và 4 ngày (vượt biên trên). Trước mỗi ca, kho Async Storage được xóa để các ca độc lập nhau; lời gọi tới gateway được thay bằng kết quả giả lập khi `NODE_ENV = test`. Mức thứ hai là kiểm thử chấp nhận thử công trên điện thoại thật theo danh sách kiểm tra. Bảng 4.6, 4.7 và 4.8 trình bày các ca kiểm thử cho ba nhóm chức năng quan trọng nhất.

Bảng 4.6: Ca kiểm thử chức năng tạo yêu cầu đặt phòng

Mã	Đầu vào / bước	Kết quả mong đợi	Kết quả
TC-01	User đặt A101 ngày mai 08:00–09:00, mục đích “Họp nhóm đồ án”	Tạo booking PENDING, Admin nhận thông báo	Đạt
TC-02	User khác đặt A101 cùng ngày 08:30–09:30	Từ chối: “Phòng đã có yêu cầu khác...” (BR-05)	Đạt
TC-03	Đặt A101 sau 4 ngày với cấu hình mặc định	Từ chối: “Chỉ được đặt trước từ 1 đến 3 ngày.” (BR-03)	Đạt
TC-04	Tạo bảo trì A101 08:00–09:00, đặt 08:15–08:45	Từ chối do trùng lịch bảo trì (BR-04)	Đạt
TC-05	Đổi cấu hình tối đa 5 ngày, đặt B202 sau 4 ngày	Chấp nhận, trạng thái PENDING (cấu hình có hiệu lực)	Đạt

Bảng 4.7: Ca kiểm thử chức năng duyệt yêu cầu và tạo mã PIN

Mã	Đầu vào / bước	Kết quả mong đợi	Kết quả
TC-06	Admin tạo mã cho booking còn PENDING	Từ chối: “Chỉ tạo mã cho yêu cầu đã được duyệt.”	Đạt
TC-07	Duyệt booking A101, Admin tạo mã	Mã 6 chữ số; validFrom = bắt đầu – 10 phút, validUntil = kết thúc + 10 phút	Đạt
TC-08	Duyệt booking phòng khóa số	<code>hasRoomPinPermission(user A101) = true</code>	Đạt
TC-09	Admin thu hồi quyền, User tạo mã	Từ chối: “Quyền tự tạo mã tại phòng này đã bị thu hồi...”	Đạt
TC-10	Cấp lại quyền, User tạo mã	Tạo thành công, <code>createdBy = user</code>	Đạt
TC-11	Chuyển SmartLock sang phòng khác rồi tạo mã cho A101	Từ chối: “Phòng chưa được gắn với SmartLock...”	Bổ sung 30/09

Bảng 4.8: Ca kiểm thử chức năng quản trị và phân quyền

Mã	Đầu vào / bước	Kết quả mong đợi	Kết quả
TC-12	Tài khoản user gọi tạo tài khoản, tạo phòng	Từ chối “không có quyền”	Đạt
TC-13	Admin khóa teacher01, đăng nhập lại; mở khóa, khôi phục mật khẩu	Khóa: đăng nhập thất bại; sau khôi phục: đăng nhập bằng mật khẩu mới	Đạt
TC-14	Admin tạo, sửa, xóa phòng C303	Tên đổi thành C304; phòng biến mất sau khi xóa	Đạt
TC-15	Hẹn nhận khóa B202 lúc 14:00, User ủy quyền “Nguyễn Văn B”	Lưu địa điểm hẹn và mã sinh viên người nhận hộ	Đạt
TC-16	Đổi booking A101 sang B202 (khác loại khóa), rồi sang A201	Lần 1 bị từ chối “cùng loại khóa”; lần 2 thành công, lịch sử đổi phòng có 1 bản ghi	Đạt
TC-17	Đăng nhập admin/1, user/2; sai mật khẩu; đăng ký trùng tên	Mở đúng chế độ; trả null; báo “Tên đăng nhập đã tồn tại.”	Đạt

Tổng hợp lại, bộ kiểm thử tự động hiện có 14 ca trong bốn tệp, bao phủ 17 kịch bản ở các bảng trên. Biên bản kiểm tra ngày 26/09/2026 ghi nhận 13/13 ca đạt tại thời điểm đó, cùng với ESLint và trình biên dịch TypeScript không báo lỗi; ca TC-11 kiểm tra liên kết SmartLock được bổ sung ngày 30/09/2026 khi hoàn thiện tích hợp khóa. Ở mức thiết bị, danh sách kiểm tra 20 mục trên Samsung M21 gồm khởi động lạnh không cần Metro, điều hướng theo vai trò, CRUD phòng, bộ chọn

tầng, luồng chọn phòng – nhập mục đích – gửi trên một màn hình, tách màn lịch chỉ đọc với màn thao tác, tạo mã do Admin, cấp và thu hồi quyền theo phòng đều đạt; nhật ký logcat không ghi nhận lỗi nghiêm trọng. Hạn chế hiện tại là dịch vụ Java chưa có bộ kiểm thử tự động riêng; các quy tắc phía máy chủ được viết tương ứng một – một với tầng dịch vụ TypeScript đã kiểm thử và sẽ được bổ sung kiểm thử JUnit ở giai đoạn đồng bộ.

4.5 Triển khai

Mô hình triển khai thử nghiệm gồm ba nút. Nút thứ nhất là điện thoại Samsung SM-M215F chạy Android, cài gói `com.bookingclassroom` phiên bản 1.0 từ APK Release; bản cài đặt đã tắt cờ debug và được ký bằng khóa demo của dự án. Ở chế độ mặc định `ENABLE_REMOTE_SYNC = false`, toàn bộ tài khoản, phòng, booking, quyền, thông báo và liên kết SmartLock được lưu cục bộ bằng Async Storage và giữ nguyên sau khi tắt mở ứng dụng. Nút thứ hai là máy tính trong cùng mạng Wi-Fi chạy gateway ở cổng 8135 và, khi bật đồng bộ, chạy dịch vụ Java bằng lệnh `mvn spring-boot:run` tại cổng 8080 với tệp dữ liệu `./data/booking`. Nút thứ ba là broker OneIoT tại `oneiot.com.vn:2111` và khóa DLWA12 kết nối Wi-Fi.

Việc cài đặt được kiểm chứng bằng lệnh `adb install -r` và bằng cách gửi trực tiếp tệp APK sang điện thoại. Sau khi xóa quy tắc `adb reverse` và dừng hẳn ứng dụng, bản Release khởi động lại bình thường với cả hai tài khoản mẫu. Mã băm SHA-256 của APK được lưu trong `release/SHA256.txt` để người nhận kiểm tra tính toàn vẹn. Hướng dẫn cài đặt chi tiết và danh sách API được trình bày tại Phụ lục B.

Chương này đã trình bày kiến trúc ba thành phần, thiết kế gói của mười module, thiết kế lớp của dịch vụ Java, cơ sở dữ liệu tám bảng, giao diện theo vai trò, kết quả xây dựng, các ca kiểm thử và mô hình triển khai trên thiết bị thật. Một số giải pháp kỹ thuật đáng chú ý được nhắc tới trong chương, gồm chống trùng lịch khi truy cập đồng thời, cấp quyền tạo mã theo phòng tích hợp khóa thông minh và kiến trúc hai chế độ dữ liệu, sẽ được phân tích chi tiết trong Chương 5.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày thiết kế và kết quả xây dựng RoomHub ở mức tổng thể. Chương này đi sâu vào ba giải pháp mà nhóm đánh giá là đóng góp chính của bài tập lớn: (i) bộ quy tắc chống trùng lịch dựa trên phép giao khoảng thời gian kết hợp khóa bí quan trong giao dịch, (ii) mô hình cấp quyền tạo mã PIN theo cặp người dùng – phòng tích hợp khóa thông minh qua gateway MQTT, và (iii) kiến trúc module cho phép cùng một bộ quy tắc chạy ở chế độ cục bộ hoặc đồng bộ máy chủ. Mỗi giải pháp được trình bày theo trình tự vấn đề, giải pháp và kết quả.

5.1 Chống trùng lịch bằng phép giao khoảng thời gian và khóa bí quan

5.1.1 Vấn đề

Yêu cầu cốt lõi của hệ thống đặt phòng là không để hai booking đang hoạt động chiếm cùng một phòng trong khoảng thời gian chồng nhau. Vấn đề này có hai mặt. Mặt thứ nhất là *logic*: cần một điều kiện chính xác để xác định hai khoảng thời gian có giao nhau hay không, xử lý đúng các trường hợp chạm biên như booking 08:00–09:00 và 09:00–10:00, đồng thời áp dụng cho cả lịch bảo trì và giới hạn theo người dùng. Mặt thứ hai là *tương tranh*: thao tác đặt phòng có dạng “kiểm tra rồi hành động” (check-then-act). Nếu hai giảng viên cùng gửi yêu cầu cho phòng A101 lúc 08:00 trong cùng một thời điểm, cả hai giao dịch có thể cùng đọc thấy phòng trống, cùng vượt qua kiểm tra và cùng chèn booking, dẫn tới race condition mà tài liệu nghiệp vụ yêu cầu phải chống [1]. Ngoài ra, giữa thời điểm tạo yêu cầu và thời điểm duyệt, tình trạng phòng có thể thay đổi do Admin tạo lịch bảo trì hoặc duyệt yêu cầu khác.

5.1.2 Giải pháp

Mô hình khoảng nửa mở. Mỗi booking trong ngày d được biểu diễn bằng khoảng nửa mở $[s, e)$ với $s < e$. Theo đại số khoảng của Allen [21], hai khoảng $[s_1, e_1)$ và $[s_2, e_2)$ cùng ngày *giao nhau* khi và chỉ khi không xảy ra một trong hai quan hệ “trước” hoặc “sau”, tức là

$$\text{overlap}([s_1, e_1), [s_2, e_2)) \iff (d_1 = d_2) \wedge (s_1 < e_2) \wedge (s_2 < e_1). \quad (5.1)$$

Điều kiện (5.1) gộp đủ chín trong mười ba quan hệ của Allen có phần giao khác rỗng (chồng một phần, nằm trong, chứa, bắt đầu cùng, kết thúc cùng, bằng nhau và các quan hệ ngược) vào hai phép so sánh, đồng thời loại trừ bốn quan hệ còn lại là trước, sau và chạm biên (*meets*, *met-by*) nhờ dùng bất đẳng thức chặt. Do giờ được lưu dạng chuỗi HH:mm có đệm số 0, phép so sánh từ điển `compareTo` cho cùng

kết quả với so sánh thời gian nên không cần phân tích chuỗi trong vòng lặp. Cùng một hàm được dùng cho ba quy tắc: trùng phòng (BR-05), trùng lịch của cùng User (BR-05) và trùng lịch bảo trì (BR-04), như Đoạn mã 5.1.

```

1 private boolean overlaps (Domain.Booking booking, String date,
2                           String startTime, String endTime) {
3     return booking.date.equals (date)
4         && startTime.compareTo (booking.endTime) < 0
5         && endTime.compareTo (booking.startTime) > 0;
6 }
7
8 private boolean hasMaintenanceConflict (String roomId,
9     String date, String startTime, String endTime) {
10    return maintenance().stream().anyMatch (item ->
11        item.cancelledAt == null
12        && roomId.equals (item.roomId)
13        && date.equals (item.date)
14        && startTime.compareTo (item.endTime) < 0
15        && endTime.compareTo (item.startTime) > 0);
16 }
17
18 private boolean isStillActive (Domain.Booking booking) {
19     return ("PENDING".equals (booking.status)
20         || "APPROVED".equals (booking.status))
21         && LocalDateTime (booking.date, booking.endTime)
22             .isAfter (LocalDateTime.now (ZONE));
23 }

```

Đoạn mã 5.1: Hàm giao khoảng và kiểm tra xung đột bảo trì trong BookingService

Thứ tự kiểm tra. Hàm `validateBooking` áp dụng các quy tắc theo thứ tự chi phí tăng dần và theo mức độ hữu ích của thông điệp cho người dùng, được tóm tắt trong Thuật toán 1. Các kiểm tra rẻ và không phụ thuộc dữ liệu khác (phòng tồn tại, định dạng ngày giờ, khoảng đặt trước) được thực hiện trước; các kiểm tra phải duyệt danh sách (bảo trì, trùng phòng, trùng User, giới hạn số yêu cầu) thực hiện sau trên tập booking *còn hiệu lực*, tức là đang PENDING hoặc APPROVED và chưa qua giờ kết thúc. Việc lọc trước tập còn hiệu lực giúp booking cũ đã sử dụng không làm sai lệch giới hạn hai yêu cầu mỗi người.

Thuật toán 1: Tạo yêu cầu đặt phòng an toàn khi truy cập đồng thời

Input: tài khoản a , yêu cầu $r = (room, d, s, e, purpose)$

Output: booking mới ở trạng thái PENDING hoặc ngoại lệ kèm lý do

if $a.role \neq user$ **or** $r.requester \neq a.username$ **then**

 ném ForbiddenException;

end

bắt đầu giao dịch; khóa ghi toàn bộ bản ghi booking

 (PESSIMISTIC_WRITE);

$B \leftarrow$ danh sách booking; kiểm tra phòng AVAILABLE, định dạng d, s, e và

$s < e$;

if $\Delta_{ngày}(hôm nay, d) \notin [minAdvance, maxAdvance]$ **then**

 ném lỗi BR-03;

end

if $\exists m \in Maintenance$ chưa hủy của phòng: $overlap(m, r)$ **then**

 ném lỗi BR-04;

end

$A \leftarrow \{b \in B \mid b.status \in \{PENDING, APPROVED\} \wedge b.end > now\}$;

if $\exists b \in A: b.room = r.room \wedge overlap(b, r)$ **then**

 ném lỗi trùng phòng (BR-05);

end

if $\exists b \in A: b.requester = a \wedge overlap(b, r)$ **then**

 ném lỗi trùng lịch cá nhân (BR-05);

end

if $|\{b \in A \mid b.requester = a\}| \geq maxActive$ **or** $purpose$ rỗng **then**

 ném lỗi BR-06;

end

lưu booking PENDING; tạo thông báo cho Admin; xác nhận giao dịch (nhả khóa);

Khóa bí quan trong giao dịch. Để phá vỡ race condition, phương thức createBooking được đánh dấu @Transactional và ngay đầu giao dịch phát câu truy vấn select b.id from Booking b với LockModeType.PESSIMISTIC_WRITE (Đoạn mã 5.2). Hibernate dịch câu lệnh này thành SELECT ... FOR UPDATE; giao dịch thứ hai muốn tạo booking sẽ phải chờ khóa của giao dịch thứ nhất được nhả khi xác nhận, và khi đó đọc được booking vừa chèn nên bị từ chối bởi BR-05. Như vậy hai bước kiểm tra và ghi trở thành một thao tác nguyên tử theo nghĩa khả tuần tự hóa [8]. Ở bước duyệt,

reviewBooking khóa riêng bản ghi đích bằng `em.find(Booking.class, id, PESSIMISTIC_WRITE)` rồi kiểm tra lại trạng thái PENDING, giờ bắt đầu, lịch bảo trì và trùng với booking APPROVED khác (BR-07); nhờ vậy hai quản trị viên bấm duyệt cùng lúc không thể xử lý một yêu cầu hai lần và yêu cầu đã lỗi thời không được duyệt nhầm.

```

1 @Transactional
2 public BookingResult createBooking(Domain.Account actor,
3                                   BookingInput input) {
4     if (!"user".equals(actor.role)) {
5         throw new ForbiddenException("...");
6     }
7     // SELECT ... FOR UPDATE: cac giao dich xep hang tai day
8     em.createQuery("select b.id from Booking b")
9         .setLockMode(LockModeType.PESSIMISTIC_WRITE)
10        .getResultList();
11    List<Domain.Booking> bookings = bookings();
12    validateBooking(actor.username, input, bookings); // BR
13    Domain.Booking booking = new Domain.Booking();
14    booking.id = "booking-" + UUID.randomUUID();
15    booking.status = "PENDING";
16    // ... gan cac truong con lai tu input
17    em.persist(booking);
18    Domain.Notification n = storeNotification("admin", "...", "
19    ...");
20    return new BookingResult(booking, n);
21 }
    
```

Đoạn mã 5.2: Tuần tự hóa thao tác tạo booking bằng khóa bí quan

Ba lớp phòng vệ. Toàn bộ giải pháp tạo thành ba lớp. Lớp thứ nhất ở giao diện: sơ đồ tầng dùng cùng điều kiện (5.1) để tô màu “đã đặt” và không cho gửi nếu phòng không trống, giúp người dùng tránh lỗi từ sớm. Lớp thứ hai là `validateBooking` trong giao dịch có khóa khi tạo. Lớp thứ ba là kiểm tra lại khi duyệt. Hai lớp sau nằm ở máy chủ nên không thể bị vượt qua bằng cách gọi API trực tiếp.

5.1.3 Kết quả đạt được

Các ca TC-01 đến TC-05 trong Bảng 4.6 xác nhận logic trùng phòng, khoảng đặt trước, bảo trì và cấu hình động hoạt động đúng. Trường hợp chạm biên 08:00–09:00 và 09:00–10:00 được chấp nhận theo đúng điều kiện chặt. Độ phức tạp của mỗi lần kiểm tra là $O(n + m)$ với n là số booking và m là số lịch bảo trì, đủ nhanh với quy mô 56 phòng. Qua phân tích, nhóm cũng nhận thấy hai điểm cần cải tiến. Thứ nhất,

câu `SELECT . . . FOR UPDATE` chỉ khóa các dòng đã tồn tại; khi bảng booking còn rỗng, hai giao dịch đầu tiên chưa bị tuần tự hóa (hiện tượng phantom). Thứ hai, khóa toàn bảng làm các yêu cầu ở những phòng khác nhau cũng phải chờ nhau. Giải pháp đề xuất là khóa bản ghi phòng bằng `em.find(Room.class, roomId, PESSIMISTIC_WRITE)`: bản ghi phòng luôn tồn tại nên loại bỏ phantom, đồng thời chỉ tuần tự hóa các yêu cầu trên cùng một phòng. Chỉ tiêu kiểm chứng đã đề ra là trong 20 yêu cầu đồng thời vào cùng phòng và khung giờ, tối đa một yêu cầu được chấp nhận; phép đo này sẽ được thực hiện khi bật chế độ đồng bộ máy chủ.

5.2 Cấp quyền tạo mã PIN theo phòng và tích hợp khóa thông minh

5.2.1 Vấn đề

Theo tài liệu nghiệp vụ, với phòng dùng pass, nhân viên CSVC phải lên tận nơi đặt pass trước giờ sử dụng và đổi pass sau khi kết thúc, vì khóa cũ không có kết nối từ xa [1]. Quy trình này tốn nhân lực và dễ sai sót, pass dùng chung có thể bị lộ cho người không còn quyền sử dụng phòng. Khi chuyển sang khóa thông minh có thể nhận mật khẩu tạm thời từ xa, xuất hiện ba câu hỏi thiết kế. Một là ai được quyền tạo mã: nếu chỉ Admin thì vẫn tạo nút thất, nếu cho mọi User thì vi phạm nguyên tắc đặc quyền tối thiểu. Hai là mã có hiệu lực trong bao lâu và bị vô hiệu khi nào. Ba là làm thế nào để ứng dụng di động không phải giữ thông tin xác thực của nền tảng IoT và không gửi mật khẩu dạng rõ qua mạng.

5.2.2 Giải pháp

Quyền theo cặp người dùng – phòng. Nhóm tách *vai trò* (Admin, User) khỏi *quyền trên tài nguyên*. Thực thể `PinPermission` lưu cặp (username, roomId) cùng cờ active và lịch sử cấp, thu hồi. Khi Admin duyệt một booking tại phòng `PIN_CODE`, hàm `grantPin` thực hiện thao tác upsert: tìm bản ghi của cặp tương ứng, nếu chưa có thì tạo mới, nếu đã bị thu hồi thì kích hoạt lại. Admin có thể thu hồi quyền theo từng phòng trong màn “Quản lý user” mà không động tới vai trò tài khoản. Khi tạo mã, hệ thống yêu cầu đồng thời bốn điều kiện: người gọi là chủ booking (hoặc Admin), booking ở trạng thái `APPROVED`, phòng dùng khóa mã số và tồn tại `PinPermission` đang hiệu lực cho đúng phòng đó.

Cửa sổ hiệu lực có đệm. Với booking $[s, e)$ và khoảng đệm g do Admin cấu hình (mặc định 10 phút), mã PIN chỉ có hiệu lực trong

$$[t_{\text{from}}, t_{\text{until}}] = [s - g, e + g]. \quad (5.2)$$

Khoảng đệm cho phép giảng viên đến sớm chuẩn bị và ra muộn thu dọn. Hàm `getPinDisplayStatus` suy ra trạng thái hiển thị của mã từ thời điểm hiện tại:

PENDING nếu $t < t_{\text{from}}$, ACTIVE nếu nằm trong khoảng (5.2), EXPIRED nếu $t > t_{\text{until}}$, và REVOKED nếu booking bị hủy hoặc mã có dấu thời gian thu hồi. Khi hủy booking hoặc đổi phòng, trường `revokedAt` được ghi nhận và quyền ở phòng cũ không còn tác dụng với booking đó (Hình 2.5). Quan trọng hơn, khoảng thời gian (5.2) được gửi xuống khóa dưới dạng Unix time nên chính khóa tự từ chối mã ngoài khung giờ, khắc phục hạn chế “hệ thống không được tuyên bố pass tự hết hạn” của khóa cũ (BR-12 trong tài liệu nghiệp vụ).

Cấp phát vị trí mật khẩu trên khóa. Khóa DLWA12 lưu mật khẩu tạm thời trong các ô đánh số 1 đến 255. Module `smart_lock` quản lý một khóa duy nhất cùng phòng đang gắn khóa và bộ đếm `nextPasswordId`. Mỗi lần tạo mã, hàm `reserveSmartLockPasswordId` kiểm tra phòng của booking đúng là phòng đang gắn khóa, lấy giá trị hiện tại rồi tăng bộ đếm theo vòng tròn (sau 255 quay về 1). Cách cấp phát vòng tròn bảo đảm các mã gần nhau không ghi đè lên nhau trong khi không cần truy vấn trạng thái ô nhớ của khóa.

Gateway tách biệt thông tin xác thực. Ứng dụng không kết nối trực tiếp với OneIoT mà gọi POST `/api/lock/temp-password` tới gateway với nội dung gồm mã, `passwordId` và khoảng hiệu lực. Gateway là nơi duy nhất giữ token OneIoT (được nhập tay vào tệp cấu hình, không đưa lên kho mã nguồn). Gateway mã hóa mã bằng AES-256 chế độ CTR [16], trong đó khóa 256 bit được dẫn xuất bằng SHA-256 từ định danh thiết bị và giá trị khởi tạo được dẫn xuất từ bộ ba (thời điểm bắt đầu, thời điểm kết thúc, `passwordId`), sau đó đóng gói vào trait `traitCreateTmpPasswordLock`, bọc trong tài nguyên `m2m:cin` của `oneM2M` và phát lên chủ đề yêu cầu với QoS 1. Gateway trả mã 202 kèm `requestId`; chỉ khi nhận phản hồi này ứng dụng mới lưu `TemporaryPin` vào booking, bảo đảm dữ liệu trong hệ thống không ghi nhận mã chưa được gửi tới khóa. Đoạn mã 5.3 trích phần tính toán chính của dịch vụ tạo mã.

```

1  const start = toLocalDateTime(booking.date, booking.startTime);
2  const end = toLocalDateTime(booking.date, booking.endTime);
3  if (end <= new Date()) throw new Error('...');
4  const configuration = await getConfiguration();
5  const code = generateSixDigitPin(); // 100000..999999
6  const validFrom = addMinutes(start, -configuration.
    pinGraceMinutes);
7  const validUntil = addMinutes(end, configuration.
    pinGraceMinutes);
8  const { lock, passwordId } =
9    await reserveSmartLockPasswordId(booking.roomId);

```

```

10 const command = await sendTemporaryPasswordToGateway(lock, {
11   roomId: room.id, roomName: room.name, code, passwordId,
12   startTime: Math.floor(validFrom.getTime() / 1000), // Unix
13   endTime: Math.floor(validUntil.getTime() / 1000),
14 });
15 const updated = await saveTemporaryPin(bookingId, {
16   code, createdAt: new Date().toISOString(),
17   createdBy: actorUsername,
18   validFrom: validFrom.toISOString(),
19   validUntil: validUntil.toISOString(),
20   lockPasswordId: command.passwordId,
21   lockCommandId: command.requestId,
22   lockDeliveredAt: command.publishedAt,
23 });

```

Đoạn mã 5.3: Tính khoảng hiệu lực và gửi lệnh tạo mã (temporaryPinService.ts, rút gọn)

Ở phía máy chủ Java, saveTemporaryPin kiểm tra lại toàn bộ điều kiện thay vì tin dữ liệu từ ứng dụng: mã phải gồm 4 đến 12 chữ số theo biểu thức chính quy `\d{4,12}`, passwordId trong khoảng 1 đến 255, phiên sử dụng chưa kết thúc, booking chưa có mã đang hiệu lực, và người gọi không phải Admin thì phải có `PinPermission` đang hoạt động. Khoảng hiệu lực cũng được máy chủ tự tính lại từ cấu hình theo (5.2).

5.2.3 Kết quả đạt được

Giải pháp loại bỏ bước nhân viên lên tận khóa đối với phòng đã gắn khóa thông minh: sau khi được duyệt, giảng viên tự tạo mã trong vài giây và mã tự hết hiệu lực trên khóa. Các ca TC-06 đến TC-11 trong Bảng 4.7 xác nhận mã chỉ được tạo sau khi duyệt, đúng khoảng hiệu lực, quyền có thể thu hồi và cấp lại theo phòng, và lệnh chỉ được gửi cho phòng đang gắn khóa. Quyền phòng không nâng vai trò người dùng, đáp ứng nguyên tắc đặc quyền tối thiểu. Nhóm cũng ghi nhận ba điểm cần hoàn thiện về bảo mật: cấu hình demo đang tắt xác minh chứng chỉ TLS của broker (`VERIFY_TLS = False`); mã PIN đang được lưu dạng rõ trong booking để hiển thị lại cho người dùng, chưa đáp ứng khuyến nghị “không mặc định lưu pass dạng rõ” của tài liệu nghiệp vụ; và khóa AES được dẫn xuất tắt định từ định danh thiết bị theo yêu cầu của firmware, nên độ an toàn phụ thuộc vào việc giữ bí mật quy tắc dẫn xuất. Các điểm này được đưa vào hướng phát triển ở Chương 6.

5.3 Kiến trúc module hai chế độ dữ liệu

5.3.1 Vấn đề

Trong quá trình thực hiện, nhóm cần demo luồng nghiệp vụ trên điện thoại thật trước khi dịch vụ Java và hạ tầng mạng hoàn thiện; đồng thời sản phẩm cuối phải

kiểm tra quy tắc ở máy chủ để chống truy cập trái phép và trùng lịch đa thiết bị. Nếu viết hai phiên bản ứng dụng, một cho cục bộ và một cho máy chủ, mã giao diện bị nhân đôi và hai bộ quy tắc dễ lệch nhau. Nếu chỉ viết phiên bản máy chủ, việc kiểm thử và demo phụ thuộc vào mạng Wi-Fi và máy tính chạy dịch vụ.

5.3.2 Giải pháp

Nhóm áp dụng mẫu Repository [7] ở tầng dịch vụ của mỗi module và một điểm cấu hình duy nhất `runtimeFlags.ts`. Hàm `isRemoteApiEnabled()` trả về `true` khi cờ `ENABLE_REMOTE_SYNC` bật và môi trường không phải kiểm thử. Mỗi hàm repository như `createBooking`, `reviewBooking`, `saveTemporaryPin`, `authenticate` có hai nhánh: nhánh máy chủ gọi `apiRequest` với token Bearer tới API Java tương ứng; nhánh cục bộ thực thi cùng quy tắc trên kho JSON của Async Storage (Đoạn mã 5.4). Giao diện chỉ gọi hàm repository và hiển thị thông điệp của ngoại lệ, nên không thay đổi giữa hai chế độ.

```

1 export async function createBooking(
2   input: CreateBookingInput): Promise<Booking> {
3   if (isRemoteApiEnabled()) {           // che do dong bo may
4     chu Java
5     const result = await apiRequest<{ booking: Booking }>(
6       '/api/bookings', { method: 'POST', body: input });
7     return result.booking;
8   }
9   // che do cuc bo
10  await assertAccountRole(input.requesterUsername, 'user');
11  const bookings = await getBookings();
12  await validateBooking(input, bookings); // BR-01..BR-06
13  const booking: Booking = { ...input, id: `booking-${Date.now
14    ()}`,
15    status: 'PENDING', createdAt: new Date().toISOString() };
16  await writeJson(BOOKINGS_KEY, [...bookings, booking]);
17  return booking;
18 }

```

Đoạn mã 5.4: Một hàm repository hai chế độ (`bookingRepository.ts`, rút gọn)

Để hai nhánh cho cùng kết quả, dịch vụ Java được viết tương ứng một – một với tầng dịch vụ TypeScript: cùng tên ca sử dụng, cùng thứ tự kiểm tra và cùng thông điệp lỗi tiếng Việt. Bảng 5.1 đối chiếu các hàm chính. Nhờ đó thông điệp mà người dùng nhìn thấy, ví dụ “Chỉ được đặt trước từ 1 đến 3 ngày.”, là như nhau dù lỗi phát sinh ở điện thoại hay ở máy chủ. Mô hình dữ liệu cũng được giữ đồng dạng: kiểu `Booking` trong TypeScript và lớp thực thể `Booking` trong Java có cùng tên trường, cùng định dạng chuỗi ngày giờ, nên Jackson tuần tự hóa trực tiếp mà không

cần lớp chuyển đổi.

Bảng 5.1: Đối chiếu tầng dịch vụ TypeScript và dịch vụ Java

Hàm TypeScript (ứng dụng)	Phương thức Java (máy chủ)	API
authenticate	login	POST /api/auth/login
assertAccountRole	requireAccount + kiểm tra role	Bộ lọc SessionAuthFilter
createBooking, validateBooking	createBooking, validateBooking	POST /api/bookings
reviewBooking	reviewBooking	POST /api/bookings/{id}/review
grantRoomPin Permission	grantPin	(nội bộ khi duyệt)
saveTemporaryPin	saveTemporaryPin	POST /api/bookings/{id}/temporary-pin
hasMaintenance-Conflict, overlaps	hasMaintenance-Conflict, overlaps	(nội bộ)

5.3.3 Kết quả đạt được

Kiến trúc này cho phép nhóm phát hành APK Release chạy hoàn toàn độc lập trên điện thoại để demo và kiểm thử chấp nhận, trong khi mã nguồn máy chủ và các nhánh gọi API vẫn được giữ để hợp nhất với nhánh syncdata. Việc chuyển sang chế độ máy chủ chỉ cần đổi cờ và địa chỉ REMOTE_API_BASE_URL, không sửa màn hình nào. Bộ kiểm thử Jest luôn chạy trên nhánh cục bộ nhờ điều kiện NODE_ENV = test, giúp kiểm thử nhanh và không phụ thuộc mạng. Hạn chế của chế độ cục bộ là dữ liệu chỉ nằm trên một thiết bị, các thao tác được xử lý tuần tự trên thiết bị đó nên không có khóa đồng thời liên thiết bị; hạn chế này được giải quyết khi bật chế độ máy chủ với khóa bí quan đã trình bày ở mục 5.1.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

So với các công cụ đang được dùng để quản lý lịch phòng như bảng tính dùng chung hay lịch tài nguyên trực tuyến, RoomHub có ba điểm khác biệt. Thứ nhất, người dùng tìm phòng trực quan trên sơ đồ chữ U của từng tầng với màu trạng thái theo khung giờ, thay vì đọc danh sách lịch bận. Thứ hai, các quy tắc riêng của nhà trường như khoảng đặt trước, giới hạn số yêu cầu, lịch bảo trì, hạn hủy và hạn đổi phòng được kiểm tra tự động và có thể điều chỉnh ngay trong ứng dụng. Thứ ba, bước đặt phòng được nối liền với bước tiếp cận phòng: phòng khóa cơ hoặc thẻ từ có lịch hẹn và người nhận hộ, phòng khóa số có mã PIN tạm thời gửi thẳng tới khóa. Ngược lại, so với phần mềm đặt chỗ thương mại, RoomHub chưa có báo cáo thống kê, check-in bằng QR và tích hợp tài khoản trường.

Trong quá trình thực hiện, nhóm đã hoàn thành phân tích nghiệp vụ từ tài liệu quy trình phiên bản 1.1, thiết kế hướng đối tượng với các biểu đồ UML, xây dựng dịch vụ Java gồm 23 lớp và 12 điểm cuối REST, ứng dụng di động mười module với mười lăm màn hình, gateway khóa thông minh và bộ kiểm thử tự động. Ứng dụng đã được đóng gói thành APK Release và chạy độc lập trên Samsung M21. Ba đóng góp nổi bật là bộ quy tắc chống trùng lịch dựa trên phép giao khoảng thời gian kết hợp khóa bi quan, mô hình cấp quyền tạo mã theo cặp người dùng – phòng tích hợp khóa DLWA12 qua MQTT, và kiến trúc hai chế độ dữ liệu dùng chung một bộ quy tắc.

Một số phần việc chưa hoàn thành gồm: bản phát hành hiện chạy ở chế độ cục bộ, dịch vụ Java chưa được triển khai trên máy chủ cố định và chưa có kiểm thử JUnit; các trạng thái Chờ bàn giao, Đang sử dụng, No-show cùng audit log đầy đủ trong tài liệu nghiệp vụ chưa được hiện thực; mật khẩu tài khoản và mã PIN còn lưu dạng rõ; hệ thống mới quản lý một khóa thông minh. Bài học rút ra của nhóm là cần chốt mô hình dữ liệu dùng chung giữa ứng dụng và máy chủ ngay từ đầu, viết quy tắc nghiệp vụ ở tầng dịch vụ có kiểm thử trước khi làm giao diện, và luôn kiểm chứng trên thiết bị thật vì nhiều lỗi chỉ xuất hiện khi chạy bản Release không có Metro.

6.2 Hướng phát triển

Trước hết, nhóm sẽ hoàn thiện các chức năng đã có. Dịch vụ Java sẽ được bật làm nguồn dữ liệu chính và triển khai lên máy chủ với PostgreSQL; khóa bi quan được chuyển xuống mức bản ghi phòng để loại bỏ hiện tượng phantom và tăng mức song song; bổ sung kiểm thử JUnit cùng bài kiểm thử 20 yêu cầu đồng thời. Về bảo

mật, mật khẩu tài khoản sẽ được băm bằng BCrypt, phiên đăng nhập có thời hạn, mã PIN chỉ lưu dạng băm hoặc mã hóa, gateway bắt xác minh chứng chỉ TLS và chỉ nhận lệnh từ máy chủ Java có xác thực.

Tiếp theo, hệ thống sẽ được mở rộng theo đầy đủ quy trình nghiệp vụ: bổ sung check-in, check-out bằng mã QR và tự động đánh dấu no-show sau thời gian chờ; quản lý số lượng, tình trạng từng thiết bị của phòng theo các quy tắc BR-13 đến BR-16 và cảnh báo booking bị ảnh hưởng khi thiết bị hỏng; ghi audit log cho mọi thao tác bàn giao, cấp mã, hủy, đổi phòng; quản lý thẻ từ với vòng đời tại tủ, giữ chỗ, bàn giao, trả và đối soát. Ở mức trải nghiệm, nhóm dự kiến bổ sung gợi ý phòng thay thế khi phòng mong muốn đã kín, chỉ đường từ thang máy tới phòng trên sơ đồ, thống kê mức độ sử dụng phòng cho đơn vị quản lý, hỗ trợ nhiều khóa thông minh và tích hợp đăng nhập bằng tài khoản trường.

TÀI LIỆU THAM KHẢO

- [1] Nhóm RoomHub, *Tài liệu quy trình nghiệp vụ ứng dụng đặt phòng học/phòng họp, phiên bản 1.1*. Tài liệu nội bộ, ngày 18/09/2026.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [3] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman, *The Java Language Specification, Java SE 17 Edition*. Oracle America, Inc., 2021. [Online]. Available: <https://docs.oracle.com/javase/10/specs/jls/se10/html/index.html> (visited on 09/30/2026).
- [4] J. Bloch, *Effective Java*, 3rd ed. Addison-Wesley, 2018.
- [5] Spring Team, Broadcom Inc., *Spring Boot Reference Documentation, version 3.4*. [Online]. Available: <https://docs.spring.io/spring-boot/3.4/index.html> (visited on 09/30/2026).
- [6] Eclipse Foundation, *Jakarta Persistence 3.1 Specification*. [Online]. Available: <https://jakarta.ee/specifications/persistence/3.1/> (visited on 09/30/2026).
- [7] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [8] P. A. Bernstein and E. Newcomer, *Principles of Transaction Processing*, 2nd ed. Morgan Kaufmann, 2009.
- [9] H2 Group, *H2 Database Engine – Documentation*. [Online]. Available: <https://www.h2database.com/html/main.html> (visited on 09/30/2026).
- [10] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, USA, 2000.
- [11] Meta Platforms, Inc., *React Native Documentation*. [Online]. Available: <https://reactnative.dev/docs/getting-started> (visited on 09/30/2026).
- [12] Microsoft Corporation, *The TypeScript Handbook*. [Online]. Available: <https://www.typescriptlang.org/docs/handbook/intro.html> (visited on 09/30/2026).

- [13] React Native Community, *Async Storage Documentation*. [Online]. Available: <https://react-native-async-storage.github.io/async-storage/> (visited on 09/30/2026).
- [14] OASIS, *MQTT Version 5.0, OASIS Standard*, 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (visited on 09/30/2026).
- [15] oneM2M Partners, *TS-0010 MQTT Protocol Binding*. [Online]. Available: <https://www.onem2m.org/technical/published-specifications> (visited on 09/30/2026).
- [16] M. Dworkin, “Recommendation for block cipher modes of operation: Methods and techniques,” National Institute of Standards and Technology, NIST Special Publication 800-38A, 2001.
- [17] Object Management Group, *Unified Modeling Language (UML) Specification, Version 2.5.1*, 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1> (visited on 09/30/2026).
- [18] OpenJS Foundation, *Jest Documentation*. [Online]. Available: <https://jestjs.io/docs/getting-started> (visited on 09/30/2026).
- [19] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [20] Nhóm RoomHub, *Mã nguồn RoomHub – classroom-booking*. [Online]. Available: <https://github.com/xwomen1/classroom-booking> (visited on 09/30/2026).
- [21] J. F. Allen, “Maintaining knowledge about temporal intervals,” *Communications of the ACM*, vol. 26, no. 11, pp. 832–843, 1983.

PHỤ LỤC

A. ĐẶC TẢ USE CASE BỔ SUNG

Phụ lục này đặc tả các use case không được trình bày trong mục 2.3. Mã quy tắc BR tham chiếu tới Bảng 2.3.

A.1 Đặc tả use case “Hủy đặt phòng”

Bảng A.1: Đặc tả use case UC06 – Hủy đặt phòng

Tác nhân	User (chủ booking)
Tiền điều kiện	Booking ở trạng thái PENDING hoặc APPROVED và chưa đến giờ bắt đầu.
Luồng chính	1. User mở “Đặt phòng của tôi”, chọn booking và bấm hủy (rút yêu cầu). 2. Hệ thống kiểm tra người gọi là chủ booking và booking còn hoạt động. 3. Với booking APPROVED, hệ thống kiểm tra còn ít nhất 30 phút (BR-08). 4. Hệ thống chuyển CANCELLED, ghi thời điểm thu hồi cho mã PIN nếu có. 5. Hệ thống thông báo “Yêu cầu đã được hủy” cho Admin.
Luồng phát sinh	3a. Quá hạn hủy: báo “Yêu cầu đã duyệt chỉ được hủy trước ít nhất 30 phút”. 2a. Phiên đã bắt đầu: báo không thể hủy.
Hậu điều kiện	Booking CANCELLED, phòng trở lại trống trong khung giờ đó, mã PIN chuyển REVOKED.

A.2 Đặc tả use case “Hẹn lịch nhận khóa và ủy quyền nhận hộ”**Bảng A.2:** Đặc tả use case UC07 – Hẹn lịch nhận khóa và ủy quyền nhận hộ

Tác nhân	Admin, User
Tiền điều kiện	Booking APPROVED tại phòng PHYSICAL_KEY, chưa đến giờ sử dụng.
Luồng chính	- Admin chọn booking, nhập ngày, giờ và địa điểm nhận khóa/thẻ (ví dụ “Phòng trực A1”). - Hệ thống kiểm tra lịch hẹn ở tương lai và trước giờ sử dụng, địa điểm không rỗng (BR-12); lưu và thông báo “Đã có lịch nhận khóa” cho User. - Nếu không tự đến nhận, User nhập họ tên và mã sinh viên/cán bộ của người nhận hộ. - Hệ thống lưu ủy quyền và thông báo cho Admin; người nhận hộ xuất trình thẻ khi nhận.
Luồng phát sinh	2a. Phòng dùng khóa số: báo “Phòng này không dùng khóa cơ/thẻ”. 3a. Thiếu họ tên hoặc mã: báo lỗi.
Hậu điều kiện	Booking có keyPickupAppointment và tùy chọn pick-upDelegate.

A.3 Đặc tả use case “Quản lý lịch sử dụng và bảo trì”**Bảng A.3:** Đặc tả use case UC08 – Quản lý lịch sử dụng và bảo trì

Tác nhân	Admin
Tiền điều kiện	Admin đã đăng nhập.
Luồng chính	- Admin mở “Lịch phòng và bảo trì”, xem lịch các booking đã duyệt. - Admin chọn tầng, chọn phòng, nhập ngày, giờ bắt đầu, giờ kết thúc và lý do. - Hệ thống kiểm tra phòng tồn tại, khoảng giờ hợp lệ, lý do không rỗng, không trùng bảo trì khác và không trùng booking đã duyệt. - Hệ thống lưu lịch bảo trì; phòng hiển thị màu bảo trì trên sơ đồ trong khung giờ đó.
Luồng phát sinh	3a. Trùng booking đã duyệt: báo “cần đổi phòng trước khi lên lịch bảo trì”. 4a. Admin hủy lịch bảo trì: ghi cancelledAt, phòng khả dụng trở lại.
Hậu điều kiện	Các yêu cầu mới trùng khung giờ bị chặn theo BR-04.

A.4 Đặc tả use case “Cấu hình nghiệp vụ”**Bảng A.4:** Đặc tả use case UC09 – Cấu hình nghiệp vụ

Tác nhân	Admin
Tiền điều kiện	Admin đã đăng nhập.
Luồng chính	- Admin mở “Quy định đặt phòng”; hệ thống hiển thị giá trị hiện tại. - Admin sửa số ngày đặt trước tối thiểu, tối đa, số booking hoạt động tối đa mỗi User, hạn hủy, hạn đổi phòng, khoảng đệm mã số, bật/tắt thông báo. - Hệ thống kiểm tra các giá trị là số nguyên không âm, tối thiểu không lớn hơn tối đa, giới hạn booking ít nhất bằng 1. - Hệ thống lưu cấu hình; các module đọc giá trị mới ở lần kiểm tra tiếp theo.
Luồng phát sinh	3a. Giá trị không hợp lệ: hiển thị thông điệp tương ứng, không lưu.
Hậu điều kiện	Quy tắc BR-03, BR-06, BR-08, BR-09, BR-10 dùng tham số mới.

A.5 Đặc tả use case “Quản lý SmartLock”**Bảng A.5:** Đặc tả use case UC10 – Quản lý SmartLock

Tác nhân	Admin; SmartLock Gateway
Tiền điều kiện	Admin đã đăng nhập; gateway chạy trong cùng mạng.
Luồng chính	- Admin mở “Quản lý khóa”, nhập địa chỉ gateway dạng <code>http://<IP>:8135</code>. - Hệ thống chuẩn hóa địa chỉ và gọi <code>GET /api/lock/status</code> để hiển thị trạng thái kết nối OneIoT, lệnh và phản hồi gần nhất. - Admin chọn một phòng khóa số để gắn khóa DLWA12. - Hệ thống kiểm tra phòng tồn tại và dùng PIN_CODE, lưu phòng được gắn, người và thời điểm gắn.
Luồng phát sinh	1a. Địa chỉ không bắt đầu bằng <code>http/https</code> : báo lỗi. 3a. Chọn phòng khóa cơ: báo “Chỉ được gắn SmartLock vào phòng sử dụng khóa mã số”. 4a. Admin tháo khóa: xóa liên kết, mọi yêu cầu tạo mã bị chặn.
Hậu điều kiện	Chỉ booking của phòng đang gắn khóa mới được gửi lệnh tạo mật khẩu.

A.6 Đặc tả use case “Quản lý tài khoản và quyền theo phòng”**Bảng A.6:** Đặc tả use case UC11 – Quản lý tài khoản và quyền theo phòng

Tác nhân	Admin
Tiền điều kiện	Admin đã đăng nhập.
Luồng chính	1. Admin nhập tên đăng nhập, mật khẩu ban đầu, mã khôi phục và bấm “Thêm tài khoản”. 2. Hệ thống kiểm tra tên 3–30 ký tự thường, số, dấu chấm, gạch ngang; mật khẩu và mã khôi phục ít nhất 4 ký tự; tên chưa tồn tại. 3. Admin tìm tài khoản, khóa/mở, đặt lại mật khẩu hoặc xóa. 4. Admin xem các phòng mà User được tự tạo mã và thu hồi quyền theo từng phòng.
Luồng phát sinh	3a. Tự khóa hoặc tự xóa tài khoản đang đăng nhập: bị chặn. 3b. Tài khoản còn booking hoạt động: không cho xóa.
Hậu điều kiện	Tài khoản bị khóa không đăng nhập được; quyền bị thu hồi chặn tạo mã theo BR-11.

B. DANH SÁCH API, CÀI ĐẶT VÀ MINH CHỨNG KIỂM THỬ

B.1 Danh sách REST API của dịch vụ Java

Mọi điểm cuối trừ đăng nhập yêu cầu tiêu đề `Authorization: Bearer <token>`. Lỗi nghiệp vụ trả về mã 400, thiếu hoặc sai token trả 401, không đủ quyền trả 403, thân phản hồi có dạng `{ "message": "..."}.`

Bảng B.1: Danh sách REST API của booking-server

Phương thức	Đường dẫn	Chức năng
POST	/api/auth/login	Đăng nhập, trả token, username, role
GET	/api/rooms	Danh sách phòng theo tầng, tên
GET	/api/configuration	Tham số nghiệp vụ hiện hành
GET	/api/maintenance	Danh sách lịch bảo trì
GET	/api/bookings	Danh sách booking, mới nhất trước
POST	/api/bookings	Tạo yêu cầu đặt phòng (User)
POST	/api/bookings/{id}/review	Duyệt hoặc từ chối (Admin), thân <code>{"decision": "APPROVED" "REJECTED"}</code>
POST	/api/bookings/{id}/temporary-pin	Lưu mã PIN tạm thời đã gửi tới khóa
GET	/api/pin-permissions	Quyền tạo mã (Admin xem tất cả, User xem của mình)
GET	/api/notifications	Thông báo của người đang đăng nhập
POST	/api/notifications/read	Đánh dấu tất cả đã đọc
POST	/api/notifications	Tạo thông báo cho chính mình

Gateway khóa cung cấp thêm hai điểm cuối: GET /api/lock/status trả trạng thái kết nối OneIoT, lệnh và phản hồi gần nhất; POST /api/lock/temp-password nhận {roomId, roomName, code, passwordId, startTime, endTime} và trả mã 202 kèm {accepted, trait, requestId, passwordId, publishedAt}.

B.2 Hướng dẫn cài đặt và chạy thử

Cài bản dùng ngay. Tải mã nguồn và cài APK có sẵn lên điện thoại Android đã bật gỡ lỗi USB:

```
git clone https://github.com/xwomen1/classroom-booking.git
cd classroom-booking
adb devices -l
```

```
adb install -r \
  ./code/booking_classroom/release/booking_classroom-v1.0.apk
```

Có thể gửi trực tiếp tệp APK sang điện thoại và cho phép cài ứng dụng không rõ nguồn gốc. Tài khoản mẫu: quản trị viên admin/1, giảng viên user/2.

Chạy bản phát triển. Trong thư mục code/booking_classroom chạy npm ci, npm start; ở cửa sổ khác chạy adb reverse tcp:8081 tcp:8081 và npm run android. Kiểm tra mã nguồn bằng npm test -- --runInBand, npm run lint và npx tsc --noEmit.

Chạy dịch vụ Java. Trong thư mục code/booking-server chạy mvn spring-boot:run; dịch vụ lắng nghe cổng 8080, dữ liệu lưu tại ./data/booking, giao diện quản trị cơ sở dữ liệu ở /h2-console. Để ứng dụng dùng máy chủ, đặt ENABLE_REMOTE_SYNC = true và sửa REMOTE_API_BASE_URL trong src/core/config/runtimeFlags.ts thành địa chỉ LAN của máy chạy dịch vụ.

Chạy gateway khóa. Trong thư mục tunghv3, nhập token thử nghiệm vào ONEIOT_TOKEN của gateway_config.py, cài thư viện bằng python -m pip install -r requirements.txt rồi chạy run_gateway.ps1. Trong màn “Quản lý khóa” của ứng dụng, nhập địa chỉ http://<IP máy tính>:8135.

B.3 Biên bản kiểm tra trên thiết bị

Bảng B.2 trích biên bản kiểm tra bản Release ngày 26/09/2026 trên Samsung SM-M215F.

Bảng B.2: Trích biên bản kiểm tra bản Release trên Samsung M21

Hạng mục kiểm tra	Kết quả
Gói <code>com.bookingclassroom 1.0</code> , cờ <code>debuggable = false</code>	Xác nhận
Dừng hẳn và khởi động lạnh khi không có Metro, không có <code>adb reverse</code>	PASS
Đăng nhập Admin và điều hướng giao diện Admin	PASS
Xác thực User và điều hướng theo vai trò trên dữ liệu mới	PASS (tự động)
Quản lý tài khoản, trạng thái, mật khẩu, thu hồi quyền phòng	PASS
Form CRUD phòng và danh sách 8 tầng dạng mở rộng	PASS
Màn duyệt yêu cầu và kiểm soát truy cập	PASS
Lịch sử dụng, bảo trì với bộ chọn 8 tầng; màn cấu hình nghiệp vụ	PASS
Sơ đồ tầng 1–8 và bộ chọn ngày giờ cuộn dọc	PASS
Chọn phòng – nhập mục đích – gửi trên một màn hình	PASS
Lịch đặt phòng chỉ đọc tách khỏi màn thao tác	PASS
Admin tạo mã tạm thời; duyệt cấp quyền theo phòng; thu hồi chặn User tạo mã	PASS
Kiểm thử mã nguồn Jest; TypeScript và ESLint	13/13 PASS
Lỗi nghiêm trọng hoặc lỗi kết nối Metro trong logcat	0